

**Manual Tecnico
Análisis y Diseño De Algoritmos**

Presentado a:

Luz Enith Guerrero Mendieta

Realizado por:

**Leidy Johana Nieto
Jerónimo García Parra**



**Universidad de Caldas
2026-1**

Índice

1. Propósito.....	3
2. Resumen Ejecutivo.....	3
3. Fundamentos Teóricos.....	4
4. Arquitectura del Software.....	7
5. Diseño Algorítmico.....	13
6. Análisis de Complejidad.....	15
7. Detalles de Implementación.....	18
8. Resultados Experimentales.....	55
9. Limitaciones y Trabajo Futuro.....	61
10. Apéndices Técnicos.....	63
11. Uso de Inteligencia Artificial Generativa.....	68

1. Propósito

El presente Manual Técnico tiene un objetivo primordial, es orientar al lector para la comprensión profunda de los aspectos algorítmicos, matemáticos y de implementación de la extensión del proyecto GeoMIP/QNodes. Este documento busca que las personas pueden entender de manera detallada y completa el proyecto que se piensa realizar, detallando como una solución algorítmica de las bi-particiones $k=2$, mediante el uso de modelos geométricos y otras técnicas nos permiten llevar a un modelo generalizado para lograr las K-particiones y obtener resultados más complejos pero de mayor envergadura. Este manual sirve como guía definitiva para comprender el problema que se piensa abordar, las limitaciones que implica realizar un cambio como este, como se explica matemáticamente, la estructura utilizada para lograr el cometido así como también todo las herramientas utilizadas dentro de este proyecto.

2. Resumen Ejecutivo

El problema que nos aborda en este proyecto nos habla de cómo se debe realizar la partición de un conjunto de datos mediante sus estados en el tiempo futuro, es decir, necesitamos averiguar cómo se debe partir un sistema completo para obtener la menor pérdida posible, se deben realizar varias técnicas para obtener el resultado más adecuado para el sistema, se utilizan conceptos como los candidatos y la marginalización, conceptos que abordaré con más detenimiento más adelante. Partir un sistema conlleva un reto computacional bastante elevado y complejo, ya que dependiendo el sistema que tan grande sea tendrá una complejidad mucho mayor para nuestra máquina por el análisis que deberá realizar, ya que hablamos de complejidades como 2^n por lo que entre más crezca nuestro sistema a evaluar más costoso se vuelve, como vemos al tener una complejidad de este calibre si usamos métodos como fuerza bruta será muy complejo obtener el resultado por la demanda que necesita el proyecto, por lo que hasta ese momento se limitaba no más a realizar una bi-partición

Por eso se plantean dos soluciones fundamentales, GeoMip y Qnodes, estas dos estrategias nos permiten de manera más eficiente llegar al resultado que necesitamos, ya que GeoMip al ser una estrategia geométrica mediante el uso del Hipercubo. En lugar de generar y evaluar exhaustivamente todas las k-particiones posibles reconstruyendo el sistema mediante productos tensoriales cruzados, el algoritmo evalúa la inercia causal mediante una función de costo lo cual permite obtener la de menor costo y así obtener las menores pérdidas.

El proyecto al uso de esta técnica logra identificar k-particiones óptimas con una complejidad asintótica radicalmente inferior a la búsqueda exhaustiva o fuerza bruta, estabilizando el tiempo de cómputo en $O(n * 2^n)$ para la construcción de la tabla de transiciones, superando las limitaciones de los métodos tradicionales.

Aunque se mitiga la complejidad factorial de las agrupaciones, el algoritmo sigue limitado por la alta dimensionalidad $n > 25$, donde la construcción del espacio n-dimensional agota la memoria disponible ya que hablamos de que un sistema de 25 nodos utiliza alrededor de 30 millones de combinaciones.

3. Fundamentos Teóricos

Para definir el sistema tenemos que pensarlo así, se define como $V = \{X_1, X_2, X_3, \dots, X_n\}$ se compone de n variables discretas o nodos en un tiempo t , de ahí se denota la partición de V que se define como $P^{(K)}$ que es una descomposición del sistema en pequeños subconjuntos, que se menciona matemáticamente como $P^{(K)} = \{P_1, P_2, P_3, \dots, P_k\}$. Estas particiones deben contar con 3 principios fundamentales.

La no vacuidad, es decir que no sea un vacío, que sean mutuamente excluyentes, es decir que uno descarta la ocurrencia del otro, y por último la exhaustividad, es decir que cada sistema se han evaluado todos los elementos que lo conforman para realizar el correcto particionamiento.

Para entender más claramente estos conjuntos vamos a ejemplificar con un caso de $n = 4$ donde nuestro sistema será $V = \{A, B, C, D\}$

Una partición válida podría ser $P^{(2)} = \{\{AB\}, \{CD\}\}$ o si pasamos al mundo de las k particiones si utilizáramos una Partición de 3 sería algo así $P^{(3)} = \{\{A\}, \{BC\}, \{D\}\}$.

Habiendo explicado más detalladamente el cómo funciona el sistema y las particiones vamos ahora a entender el objetivo que buscamos con estas particiones, El objetivo es encontrar la k -partición que minimice la discrepancia estructural y causal, es decir, la pérdida de información entre el sistema que está acoplado originalmente y el sistema fragmentado. para entendernos mejor el objetivo es encontrar la Partición de Información Mínima (MIP) ϕ es identificar la forma de "cortar" el sistema que genere la menor pérdida de información causal.

La fórmula matemática que describe este proceso se muestra así:

$$\min_{p^{(k)} \in \pi(V)} \phi(P^{(k)}) EMD(p(v_{t+1}|v_t), q^{(p)}(v_{t+1}|v_t))$$

Donde aplicamos varias restricciones:

- $p^{(k)}$ debe ser una partición válida del conjunto V .
- El número de particiones k está acotado por $2 \leq k \leq \min(5, n)$
- Las distribuciones probabilísticas p y $q^{(p)}$ se mantienen en el rango entre $(0,1)$ bajo el condicionamiento del estado inicial.

Para hablar más en profundidad del hay unos conceptos claves que quiero detallar para que entendamos que se va a realizar para lograr una solución óptima para nuestro Problema.

Distancia Hamming: La distancia Hamming es simplemente cuantos pasos deben cambiar para pasar a un estado a otro, si por ejemplo tenemos un 1001, cuanto se debe cambiar

para pasar 1011, vemos que sería un paso, eso es lo que hace Hamming determinar cuántos pasos se dan para que cambie al otro estado.

Hipercubo: Es la definición de llevar un cubo que es de tres dimensiones, a las dimensiones siguientes 4, 5, etc. matemáticamente es como un mapa donde cada esquina representa una combinación posible de nuestro sistema. Esto se utiliza con la finalidad de encontrar el camino más corto para llegar de un estado a otro, esto se compagina con la distancia hamming

Marginalizar: Es una técnica matemática para eliminar el ruido y concentrarse solo en la información que importa en ese momento, simplificando el problema. Se utiliza cuando queremos evitar utilizar información irrelevante para encontrar la mejor partición.

Earth Mover's Distance (EMD): La usamos principalmente cuando queremos medir cuánta información se perdió o se modificó cuando cortamos el sistema en pedazos, es decir, cuando se realiza la partición.

Candidatos: en lugar de analizar muchas variables al mismo tiempo, el programa aísla un pequeño grupo de variables los candidatos para estudiarlas de cerca, mientras "congela" al resto del sistema para que no interfiera en la prueba, lo que nos facilita hallar la mejor partición en mejor tiempo.

Matriz de Probabilidad de Transición (TPM): La TPM es una matriz que le dice a la computadora cómo cambiará el sistema de un milisegundo al siguiente basándose en las probabilidades, esto para determinar qué tan probable se de un cambio a otro estado del sistema.

Partición de Información Mínima (MIP - Minimum Information Partition): . La MIP es el corte donde el sistema pierde menos información. Al encontrar este corte, descubrimos cuáles partes son más independientes del sistema y cuáles están profundamente conectadas entre sí.

Tensores y Descomposición Tensorial: es tomar esa enorme información que nos da esos estados y lo buscamos rebanar en láminas más delgadas y manejables para poder procesarlas sin que la computadora colapse así poder utilizar nodos extremadamente grandes para lograr solucionar y obtener la MIP.

Explosión Combinatoria: Como todo lo manejamos 2^n es tan grande manejarlo que entre más grande sea la cantidad de nodos, las combinaciones que se pueden hacer para realizar la partición son tan grandes que el sistema colapsa al no poder realizar todas esas posibilidades, por eso se utilizan todas las herramientas que acabamos de definir para evitar tener esa explosión.

Pasando ahora al entendimiento de la estrategia GeoMIP, el enfoque clasico que este maneja es para las bi-particiones donde mapea los 2^n estados binarios de un sistema a los vértices de un hipercubo de n dimensiones, traduciendo el problema probabilístico a uno geométrico basado en tensores y distancias topológicas (Hamming).

En el caso de las bi-particiones, el algoritmo GeoMIP buscaba el hiperplano de corte que dividiera el hipercubo con el menor costo de transición, o sea, el corte estructural más débil. Para extender este marco a k-particiones, el modelo geométrico se expande para evaluar cortes simultáneos a través de múltiples ejes ortogonales, el tensor de probabilidad se descompone en k tensores marginalmente independientes.

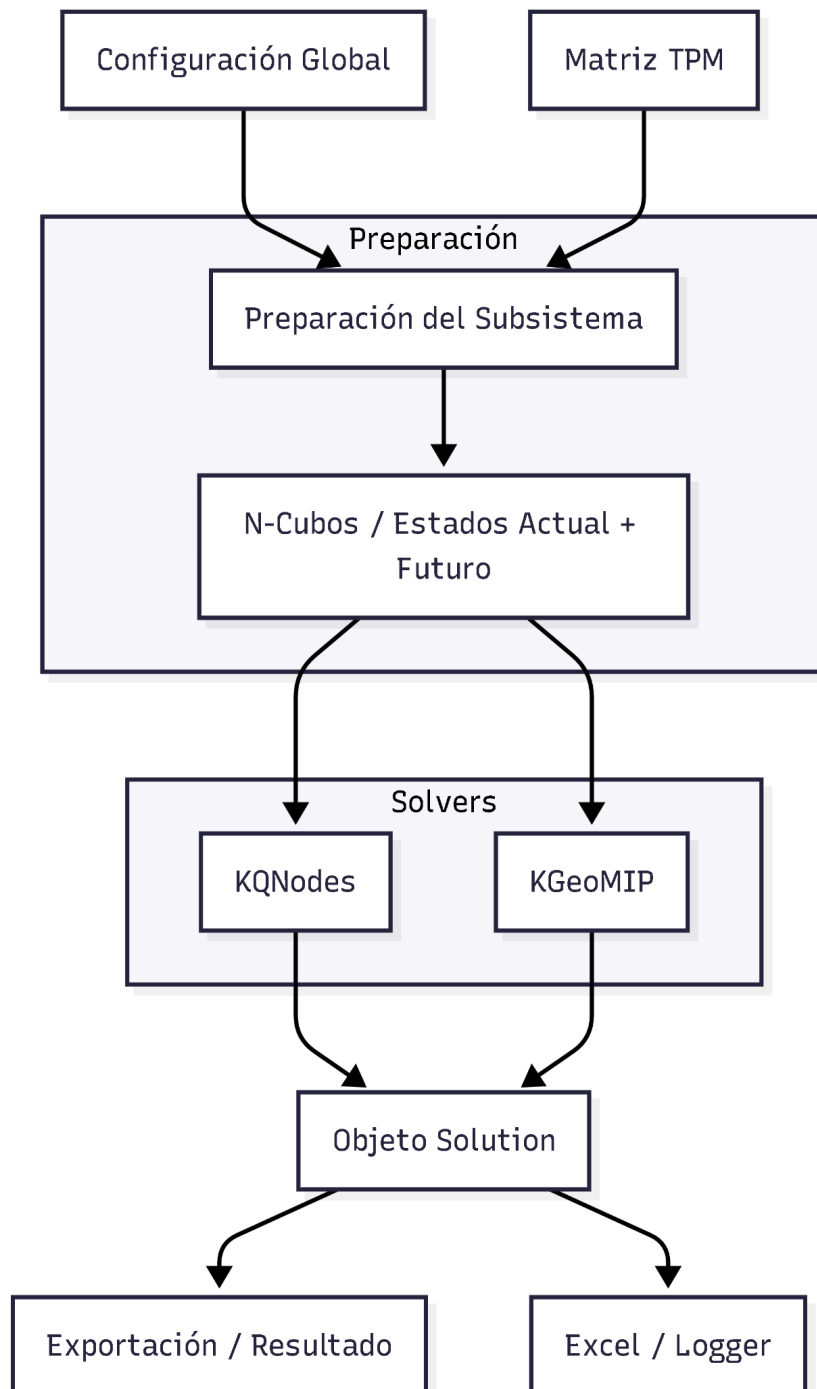
Esta estrategia puede seguir siendo aplicable y genera respuestas óptimas debido a la propiedad de aditividad estructural de los grafos causales. Al calcular los costos de transición en el nivel de hipercubo, es decir, costos nivel por nivel desde el estado actual al futuro, GeoMIP puede agrupar las afinidades topológicas de los nodos. En lugar de someter al sistema a fuerza bruta combinatoria, GeoMIP prioriza subconjuntos de nodos cuyas probabilidades de transición presenten una distancia geométrica mínima entre sí, asegurando que la partición devuelta represente el corte de menor impacto causal.

Cuando queremos pasar de bi-particiones a k-particiones, la complejidad de nuestro proyecto aumenta severamente. Por ejemplo, con $n=10$ pasamos de evaluar 511 bi-particiones a más de 115,974 formas posibles de k-particiones. Por lo tanto, herramientas clásicas basadas en fuerza bruta como PyPhi se vuelven completamente inviables, ya que el tiempo de cómputo colapsa ante la explosión combinatoria. Es aquí donde GeoMIP asume el gran reto de optimización, utilizando su mapeo geométrico para encontrar las k-particiones óptimas sin necesidad de calcular exhaustivamente todas las combinaciones posibles.

4. Arquitectura del Software

Aquí encontraremos los diagramas a nivel general del funcionamiento del proyecto, para mejor comprensión y entendimiento de nuestro problema, los diagramas pretenden ser legibles y entendibles por cualquier persona que necesite un contexto rapido de lo que se está haciendo aquí, primeramente vamos a ver la arquitectura general del proyecto.

Arquitectura del Sistema:



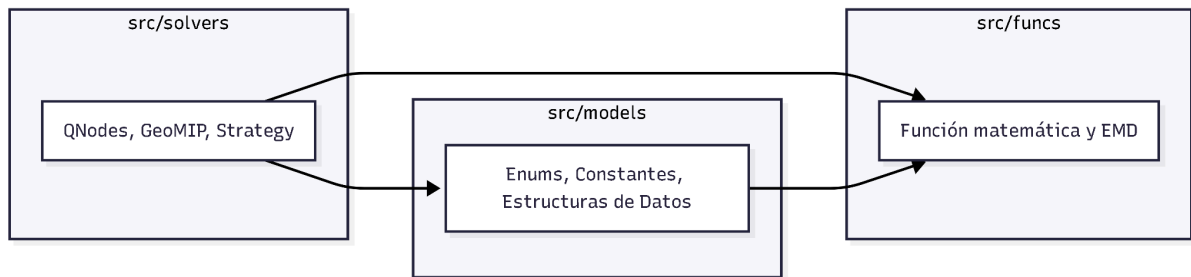
Primero tenemos la primera fase que es la entrada de datos, que sería el material que entra crudo sin modificar, El flujo comienza en la parte superior con los dos ingredientes fundamentales que el sistema necesita para funcionar, uno es la configuración global, que serían las reglas o restricciones que se tiene que cumplir, aquí definimos los parámetros de ejecución se van a utilizar, las métricas de EMD y optimizaciones de rendimiento; después entra la matriz TPM, que es uno de los componentes principales, nos representa el comportamiento de cómo el sistema va a cambiar en base a las probabilidades.

Una vez entendida la entrada de datos vamos a la fase de preparación, es la organización de los datos que vamos a usar para el correcto cálculo de las particiones que se le harán al sistema, primero en esta fase entramos en la preparación del subsistema, la idea es que no se evalúe la red completa, se tiene que recortar y asilar en un subconjunto de nodos, ya que es lo que nos interesa evaluar en ese momento. Después vamos con los estados, el alcance y mecanismo y cómo hacemos el uso de los N-cubos, o hipercubo, esto lo que hace principalmente es la traducción matemática del proceso, para darnos un resultado, la red que se aísla, se transforma en los modelos de hipercubos, donde se separan las variables que representen el presente de las que representarán el futuro.

Seguimos bajando por el flujo y nos encontramos con la capa de los solucionadores o solvers, estos funcionan como el motor de cálculo de las particiones. Cuando el subsistema se encuentra preparado matemáticamente, llega al punto clave, donde utilizamos el patrón de strategy, que nos sirve como el motor que nos ayudará a elegir la estrategia para hacer ese cálculo de la menor pérdida de información. Las dos estrategias que estamos utilizando son la versión mejorada de GeoMIP, y el QNodes, el Qnodes funciona como un algoritmo que utiliza las estrategias de aproximación y las caches de las memorias, para resolver redes grandes con una buena eficiencia en el tiempo, por su lado GeoMIP o KGeoMIP, es un enfoque distinto ya que se utiliza un enfoque geométrico profundo para hacer el cálculo de los cortes o particiones.

Por último vamos a la parte final del flujo, que es la fase donde se hace la entrega del resultado, una vez que el motor termina los cálculos de las particiones, las soluciones se empaquetan en Solution, estas soluciones primero se muestran en la consola del usuario, donde puede observar como fue la mejor partición para cada k , y dese hace el envío de esa información al excel utilizado en las pruebas que nosotros ejecutemos donde se almacenará los resultados obtenidos.

Diagrama de Paquetes:



Este diagrama representa cómo está estructurado físicamente el código del proyecto en las carpetas o como se le suele llamar los paquetes, el propósito de estos diagramas es demostrar la separación de las responsabilidades, porque un proyecto tan complejo, no puede ser trabajado todo en uno, cada componente debe realizar una tarea específica que compagina a otra tarea y lograr el mejor funcionamiento dentro de nuestro sistema.

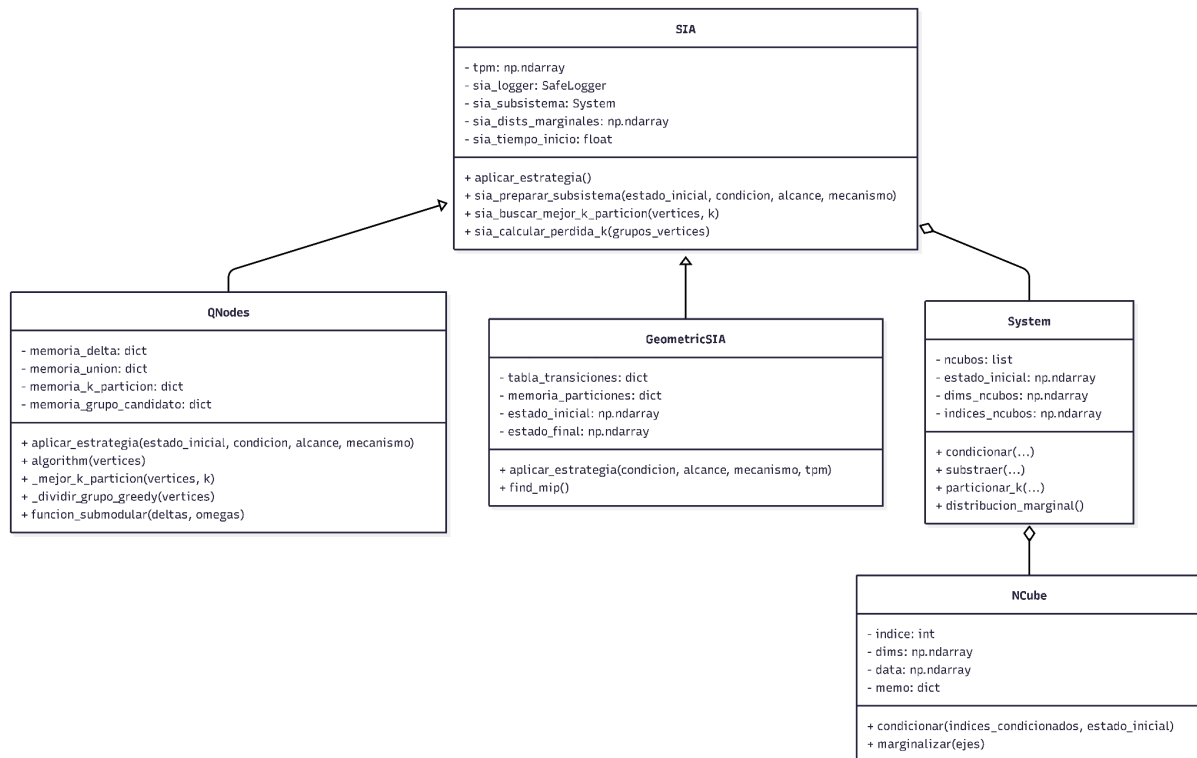
Empecemos por las funciones, estas contienen el funcionamiento matemático de nuestro sistema, aquí vemos la lógica del cálculo de la métrica como EMD, la generación de candidatos, el llamado para hacer el tratamiento de los datos en excel, entre otras funcionalidades. La idea de esta fase es que haga los cálculos matemáticos, estas funciones sirven para ser reutilizables, se pueden utilizar en otros proyectos.

Pasamos ahora con los modelos, aquí tenemos el diccionario de las variables que utilizamos y el modelo que tienen los datos, se tienen las estructuras de datos, como el funcionamiento del N-cubo, los enumeradores, y definición de algunas variables del sistema.

Estas usan las funciones para validar la información procesada, este paquete nos sirve como los planos que debe hacer el proyecto.

Por último tenemos los solucionadores, no malinterpreten el diagrama, se está manejando por estrategias, pero para dar mayor claridad de lo que significan estos son los que proporcionan las soluciones a nuestro problema tratado, estos funcionan como los orquestadores de todo el proceso que se realiza a nivel matemático, deciden cómo se debe dividir los subsistemas y determinan esa mínima pérdida de la información, los solvers o estrategias usan los otros dos paquetes para realizar su trabajo, sin esto el solucionador en sí, no nos podría dar la solución al problema.

Diagrama de Clase General:



A diferencia de los diagramas de flujo o los de paquetes, los diagramas de clase nos permiten ver una jerarquía estática y estructural del software. Funciona como el esqueleto del sistema. Muestra quién es el padre de quién y qué componentes están compaginados dentro de otros.

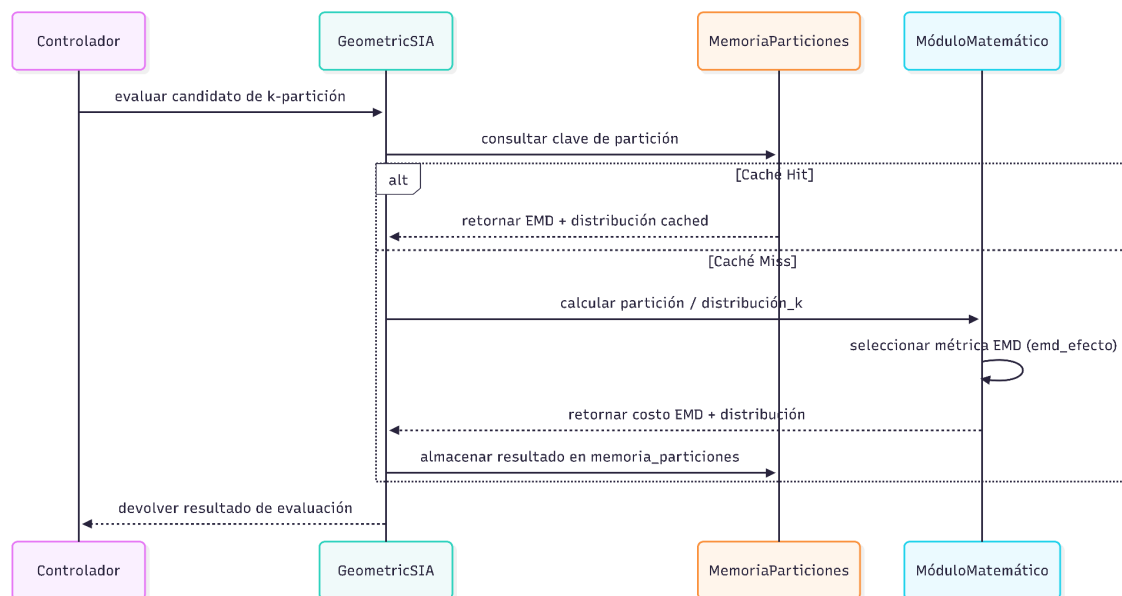
Para empezar debemos entender la jerarquía principal del sistema, las flechas que apuntan hacia arriba nos determinan e indican que es una herencia, es decir es un tipo de algo que es más grande, aquí es donde vemos ese patrón aplicado de las estrategias. Tenemos la clase grande o padre que sería nuestro SIA, funciona como esa clase abstracta que tiene unas directrices que las hijas deberán cumplir, esta SIA maneja bien los registros y la matriz de estado general, pero esta no sabe manejar el problema matemático.

Las Clases Hijas Que serían QNodes y KGeoMIP son las especialistas. Heredan todas las capacidades de SIA, pero cada una implementa el cálculo a su manera y por eso son tan diferentes unas de otras, pero son parecidas en estructura, eso es lo que hace el patrón. La estrategia geométrica encontramos tablas de transiciones que termina siendo un enfoque más analítico, Qnodes por su parte tiende a ser de diccionarios de memoria, que buscan esa optimización por el caché.

Pasando de cómo se reparten las tareas es donde existen las flechas de tipo diamante, que se leen como una composición, las composiciones, es que se tiene algo de, esto demuestra que se separan la lógica algorítmica de la gestión de los datos. La clase SIA no maneja directamente la manipulación matemática de las probabilidades. En su lugar, tiene un objeto

System. Cuando el algoritmo necesita hacer un corte, no lo hace él mismo, sino que le da la orden al System para que manipule el estado matemático. de ahí pasamos del System al NCube, debido a que System delega la estructura más pequeña, ya que se compone de múltiples NCubes, estos cubos son la representación fundamental de la memoria en la teoría de la información integrada, ya que guardan los índices y las dimensiones.

Diagrama de Secuencia KGeoMIP:



Estos diagramas ilustran la interacción dinámica entre los componentes del sistema al momento de evaluar una partición candidata. En lugar de mostrar un simple flujo lineal de cálculos, buscamos entender de forma más sencilla que es lo que ocurre internamente cuando se está realizando una prueba.

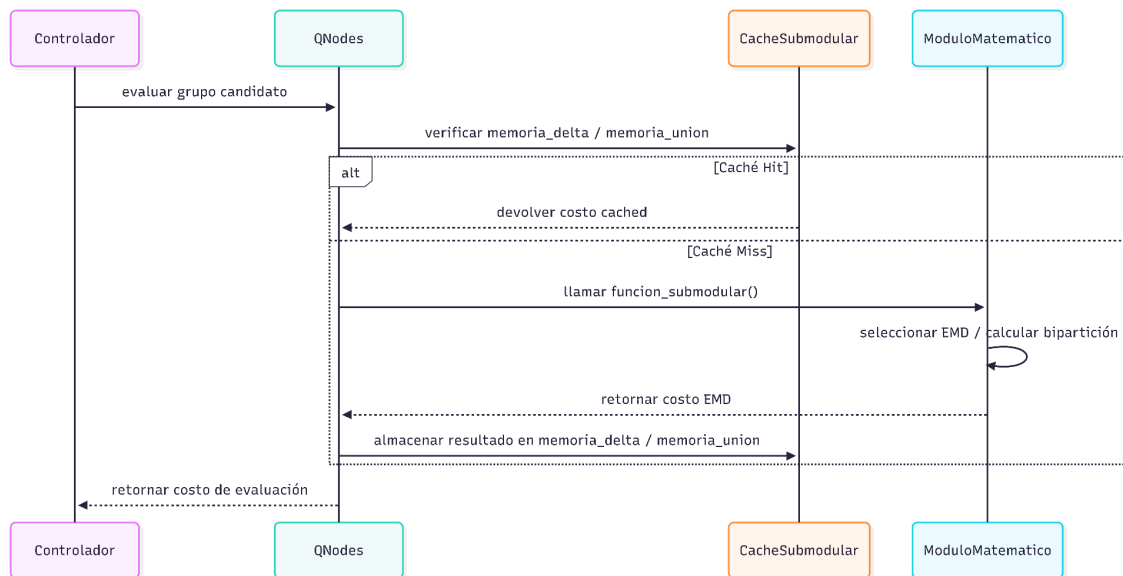
El proceso de evaluación se divide en un ciclo de consulta y bifurcación, cuando el sistema o el controlador solicita al motor geométrico evaluar el costo de realizar un corte específico en la red, el motor no realiza cálculos matemáticos de inmediato. En su lugar, genera una firma única para ese corte y consulta primero su módulo de memorización.

Después hace una bifurcación lógica, ya que aquí reside el núcleo del rendimiento del algoritmo. Aquí nos encontramos con dos escenarios. La primera nos dice que si la partición candidata ya fue analizada en una iteración previa, el sistema interpreta esa información, recupera inmediatamente la métrica EMD almacenada en la caché y la devuelve al controlador. Este es un camino más corto y nos ayuda a evitar llamar al procesador matemático, reduciendo el tiempo de ejecución a una simple búsqueda en diccionarios de orden temporal constante.

Si es la primera vez que el sistema se encuentra con esa partición, se ve obligado a utilizar el motor matemático para que ejecute el algoritmo pesado de bipartición y calcule la

distancia EMD real. Sin embargo, antes de devolver la respuesta al controlador, el sistema guarda el nuevo resultado en sus diccionarios de memoria para GeoMIP garantizando que jamás tendrá que volver a calcular este estado en el futuro.

Diagrama de Secuencia QNodes:



A diferencia del enfoque analítico tradicional, este motor ataca la explosión combinatoria de la red utilizando una estrategia voraz que se apoya en propiedades matemáticas submodulares. Este diagrama nos muestra cómo el sistema evita recalcular escenarios redundantes mediante una capa de intercepción muy agresiva.

El ciclo de evaluación para un grupo candidato se desarrolla de la siguiente manera, primero se utiliza la interceptación submodular, este proceso inicia cuando el controlador le pide a QNodes que evalúe un grupo candidato. Antes de realizar cualquier cálculo de distancias, QNodes se comunica con su gestor de la caché, aquí le pregunta a sus diccionarios si ya se tienen esas caches de tipo delta y de unión. después encontramos que el rendimiento del algoritmo se define en este recuadro de decisión, donde el sistema toma uno de dos caminos.

El primer caso es tomar el atajo computacional, que significa, si las combinaciones del grupo candidato ya existen en los diccionarios históricos, es decir, la memoria delta y unión, el sistema corta la ejecución. Recupera el costo matemático almacenado y lo devuelve al controlador en una fracción de milisegundo. Si no es el caso positivo para hacerlo rápidamente pasa a realizar el respectivo cálculo, Si los registros no contienen las piezas necesarias, el sistema asume el costo computacional. Por lo que se llama al módulo matemático a calcular la bipartición óptima y ejecutar la métrica EMD. Sin embargo, el paso más crítico ocurre justo antes de acabar la operación y es que la estrategia QNodes aprende. Toma los resultados de los deltas y las uniones y los inyecta en sus diccionarios de memoria.

5. Diseño Algorítmico

El algoritmo desarrollado implementa la estrategia KGeoMIP, es decir, elevar más el nivel para ya no solo hacer las biparticiones sino conseguir una mejora optimizada de los enfoques tradicionales de la Teoría de la Información Integrada. La filosofía de diseño se basa en la geometría del espacio de estados para evadir la explosión combinatoria propia de la búsqueda exhaustiva como la Fuerza Bruta. Mientras que los enfoques clásicos como QNodes nos brindan soluciones pero hacen evaluaciones no tan eficientes, en cambio el KGeoMIP trata al sistema como un hipercubo dimensional donde las relaciones causales se manifiestan como distancias y flujos energéticos. En lugar de fragmentar a ciegas, el algoritmo analiza el hipercubo de estados, identificando las líneas de menor resistencia causal para proponer cortes con una eficacia mucho mayor, conseguir una forma de hacer cortes donde no se vea tanta pérdida como otros, usando un rango de $k = 2$ a $k = 5$ bloques, reduciendo drásticamente el espacio de búsqueda sin sacrificar la precisión en el hallazgo de la Partición de Información Mínima para los sistemas de nuestros ejemplos.

La eficiencia del sistema empieza en la selección de estructuras de datos orientadas al acceso en tiempo constante $O(1)$ y a la vectorización de operaciones matemáticas, se utilizan los N-Cubos que nos permiten hacer una representación multidimensional de las distribuciones de probabilidad del subsistema. Sus dimensiones físicas representan los estados posibles de la red 2^n de forma que se llegue más rápidamente al corte adecuado para un sistema dado. Se hace uso de los datos planos que nos permite usar arreglos unidimensionales optimizados que se terminan derivados de los N-Cubos. Almacenan las probabilidades de transición alineadas en orden little-endian. Permiten reemplazar búsquedas en matrices multidimensionales por indexación directa mediante enteros binarios, acelerando el acceso en la CPU. Utilizamos las tablas de costos y transiciones que funcionan como una estructura de diccionario indexada por tuplas de estados origen-destino, estados iniciales y estados finales. Almacena los vectores de costo de transición acumulados para cada variable futura. El algoritmo también tiene una tabla hash que actúa como memoria caché de corto plazo. Almacena pares clave-valor donde la clave es la firma estructural de la partición y el valor contiene la tupla de EMD, haciéndonos evitar el recálculo redundante de particiones idénticas generadas por diferentes subrutinas.

Para la estrategia de Búsqueda y la selección de los Candidatos necesitamos explorar el espacio de las k-particiones de manera eficiente, el algoritmo tiene que abandonar el muestreo exhaustivo y se adopta una aproximación heurística híbrida que combina técnicas de Divide y Vencerás, Clustering Geométrico y Algoritmos Voraces, Se hace una estrategia que nos permite hacer biparticiones por complementariedad de los niveles Hamming $k = 2$, ya que se ordenan las variables del futuro según su costo acumulado en la tabla de transiciones. Mediante el uso del enfoque voraz, seleccionamos los niveles intermedios del hipercubo donde pasar de un estado a su complementario directo implique la menor pérdida de información.

Se hace un clustering por estados ancla ya que su diseño nos facilita para realizar particiones multibloque o sea cuando tenemos un $k > 2$. Se seleccionan de forma voraz los k estados dentro del hipercubo que minimicen el costo de transición y maximicen su separación espacial esto medido gracias al uso de las distancias de Hamming. Estos

estados operan como centros de gravedad; las variables del presente y futuro se asignan al bloque del ancla más cercana mediante una proyección geométrica de costo mínimo.

También utilizamos la división recursiva geométrica ya que al aplicar un enfoque de Divide y Vencerás, el algoritmo identifica el bloque con mayor densidad de variables y le aplica una bipartición local basada en costos de transición, fragmentando la estructura de manera balanceada hasta alcanzar las k partes solicitadas. Se aplica un agrupamiento por costo cero, es una técnica heurística de optimización que detecta variables independientes, queriéndonos decir que si hay un nodo donde está tan lejos que no depende de otro, estos se les agrupa de manera uniforme para evitar la ruptura de enlaces informativos reales, osea los que nos generan mayor pérdida de la información.

Para conseguir una buena evaluación de particiones, ese calculo de perdidas, cuando tengamos una k -partición candidata la sometemos a un filtro de restricciones estructurales, algo que nos permita validar dicha partición, para certificar que contenga exactamente los k -bloques y que ningún bloque carezca de elementos en el presente o futuro. Si la partición es válida, el procedimiento de evaluación recupera o calcula la distribución de probabilidad fragmentada del subsistema. El impacto o pérdida de información provocado por el corte se cuantifica mediante la función EMD, la cual mide la distancia estadística o coste de transporte matemático necesario para transformar la distribución de la partición degradada de vuelta a la distribución marginal del sistema intacto. La partición que arroje el menor valor de EMD en todo el espectro de búsqueda es proclamada como la MIP.

El programa integra tres pilares de optimización de nivel de producción para mitigar la complejidad temporal y espacial, la memoization o caching Estructural: A través de la clave de un candidato, algo que lo identifique como único, todas las propuestas de corte se ordenan y normalizan en tuplas inmutables antes de ser evaluadas. Si la firma matemática ya existe en la memoria de las particiones, la EMD se recupera en tiempo $O(1)$, omitiendo por completo el costoso cálculo algebraico de las distribuciones. Se realiza una paralelización y aislamiento de los procesos, El procesamiento de las pruebas se ejecuta delegando cada evaluación a un subproceso independiente comunicado mediante colas de memoria. Esto nos ayuda a evitar el cuello de botella del Global Interpreter Lock (GIL) de Python, se distribuye la carga equitativamente entre los núcleos del procesador y previene fugas de memoria.

Se hace un control de la concurrencia por el tiempo límite, cada proceso es monitoreado en tiempo real. Si el análisis de un estado crítico supera el umbral de seguridad establecido, el proceso padre ejecuta una terminación forzada, salvaguardando los recursos del sistema y garantizando la continuidad de la campaña de pruebas masivas sin interrupciones. Ante los archivos TPM masivos que excedan los límites físicos de la memoria RAM que provocan muchas veces cuelgues o reinicios forzados, el orquestador implementa un plan de contingencia que segmenta la lectura del archivo CSV utilizando Pandas con precisión simple float32, reduciendo la huella de memoria a la mitad y mediante el uso de los Chunks para el uso de bloques evitando colapsos del programa o computador.

6. Análisis de Complejidad

El análisis temporal del algoritmo la dividimos en dos fases críticas, la primera de todas es la inicialización del relieve geométrico, es decir, el mapeo del hipercubo que estamos utilizando y la fase iterativa de evaluación de particiones candidatas. Sean n el número de variables o como les hemos estado nombrando nodos del sistema y k el número objetivo de particiones $2 \leq k \leq 5$.

Para esta primera fase de Inicialización, se realiza primero el mapeo del Hipercubo, esta operación que se domina en esta fase es la exploración BFS por capas y la ejecución de la subrutina, el cálculo de los costos. El algoritmo itera sobre los 2^n estados que componen los vértices del hipercubo. Para cada estado origen, se evalúan sus transiciones hacia el complemento. La distancia de Hamming promedio en un hipercubo es $n/2$, lo que implica que el bucle de acumulación recursiva realiza pasos proporcionales a n . La indexación y cálculo de diferencias en los arreglos aplanados se realiza de forma vectorizada en tiempo constante para cada variable, donde se termina tomando $O(n)$ operaciones. Por lo tanto, la complejidad asintótica de la construcción de la tabla de costos se termina definiendo por $\Theta(n * 2^n)$.

Para la fase donde se explora y se intenta analizar la mínima pérdida de información, En lugar de explorar las particiones posibles a través de combinatoria de Bell lo que hacen regularmente la fuerza bruta u otras estrategias, se planteó un generador heurístico que limita los candidatos por cada nivel de k .

Para la generación de los candidatos, se utilizan las heurísticas de asignación por anclas, la división recursiva y el agrupamiento por costo cero, ya que estos nos generan un número de candidatos lineal o acotado respecto a las variables, algo que termina siendo mucho menor al número de Bell, En la práctica se termina teniendo una complejidad de $O(n^2)$ en el peor escenario de subdivisión. para nuestra evaluación algebraica de las distribuciones y EMD, hacemos que por cada candidato válido, se empieza a calcular la distribución marginal particionada requiere operar sobre tensores de tamaño de 2^n .

Para el cálculo de la Earth Mover's Distance (EMD) linealizada mediante optimización en redes para distribuciones de tamaño 2^n tiene una complejidad de corte superior de $O((2^n)^3) = O(2^{3n})$. Sin embargo, gracias al mecanismo de Memoization es decir una caché estructural, el costo real se reduce a una consulta $O(1)$ tras la primera ocurrencia de la firma estructural.

Para la complejidad temporal global y los tan vistos cuellos de botella se combinan ambas fases, la complejidad temporal total del algoritmo se expresa como:

Donde tenemos la operación dominante que sería el cálculo de la distribución marginal y la métrica EMD sobre el espacio de estados indexado. y el cuello de botella como se tiene un crecimiento exponencial del hipercubo 2^n . La heurística nos reduce drásticamente el número de particiones evaluadas, el tamaño intrínseco de los vectores de probabilidad que

representan el sistema sigue acotado por 2^n , limitando la ejecución en tiempo polinomial para sistemas donde $n > 20$ donde nuestro sistema empieza a tener carencias por lo difícil de optimizar.

Para la complejidad espacial tenemos que el uso de memoria del algoritmo se divide de forma estricta entre estructuras permanentes y dinámicas asignadas en tiempo de ejecución. Existen las estructuras permanentes o lo que serían los datos Planos, estas almacenan las distribuciones de probabilidad de los N-Cubos de forma aplanada. Cada n-cubo requiere un arreglo de tamaño 2^n . Para n variables, la ocupación en memoria es de $\Theta(n * 2^n)$ flotantes.

En la tabla de transiciones en el peor de los casos, se almacenan las relaciones e interconexiones de costo entre todos los pares ordenados de estados calculados en la BFS. Su tamaño en memoria está acotado por $O(n * 4^n)$. Esta es la estructura con mayor tasa de consumo espacial y el talón de aquiles en la explosión combinatoria.

Para las estructuras temporales y el caché de la memoria de particiones, se utiliza una tabla hash de memoization que nos almacena las firmas de las particiones evaluadas. Su tamaño máximo es proporcional a la cantidad de candidatos únicos generados por las heurísticas, es decir, $O(\sum m_k * 2^n)$, donde cada registro almacena el vector de distribución resultante de tamaño 2^n .

Se aísla por multiprocessing, ya que al lanzar subprocesos a través del multiprocessing, el sistema operativo duplica el espacio de direccionamiento de memoria. Por ende, la memoria espacial dinámica escala multiplicada por el número de núcleos activos ejecutando pruebas concurrentes. La cota asintótica fuerte para la complejidad espacial global termina siendo de $O(n * 4^n)$.

En nuestro mejor caso para las condiciones del Sistema, el sistema presenta un alto grado de modularidad o desconexión causal latente se puede decir por ejemplo, variables independientes con transiciones de costo cero hacia el estado complementario. Se tiene una dinámica algorítmica debido a que la heurística o la selección de anclas encuentran de inmediato particiones perfectas que anulan la pérdida de información $EMD = 0$. Además, la caché estructural intercepta la mayoría de las propuestas de los sub-bloques en tiempo $O(1)$ por lo que la complejidad en el Mejor Caso es un $\Theta(n * 2^n)$ para sus operaciones, dominada únicamente por la fase de inicialización geométrica, reduciendo a cero el tiempo de cómputo del resolutor EMD.

Para el peor caso posible en las condiciones del Sistema, nuestro sistema es altamente integrado, simétrico y acoplado, es decir, todas las variables influyen uniformemente sobre las demás, como si fuera una red de conectividad total. El valor de k se fija en su límite superior que es $k = 5$. En este caso particular la dinámica algorítmica no tenemos estados de ancla privilegiados ni transiciones de costo cero. Las heurísticas generan la cantidad máxima de candidatos permitidos. Ninguna partición estructural se repite, forzando fallos

constantes en la caché de memoization e induciendo al software a calcular el resolutor EMD en cada iteración sobre tensores densos. por lo que la complejidad en el peor caso termina siendo un $O(n^2 * 2^{3n})$.

Haciendo la comparativa con la fuerza bruta, tenemos una mejora considerable y más teniendo en cuenta que debemos hacer más particiones aparte de $k = 2$ por lo que considero que es un avance prometedor ya que mediante el uso de las herramientas y algoritmos que se han aplicado en nuestro sistema tenemos nos ha permitido hacer particiones de buen nivel, con buena eficiencia y adicionalmente para nodos extremadamente pesados.

7. Detalles de Implementación

Como tenemos nuestras dos estrategias separadas en dos proyectos diferentes, voy a explicar uno y después pasar a explicar el otro, aunque la estructura es parecida por lo que en la segunda estrategia se evitarán varios archivos para evitar explicaciones redundantes.

Empezamos por la estrategia que tocó implementar con más detenimiento que sería es GeoMIP.

GeoMIP tiene una carpeta llamada `Data` en esta carpeta encontramos dos elementos que es otra carpeta llamada `samples` y un archivo llamado `creation.py`.

Dentro de `samples` encontramos varios `.csv` que hacen referencia a los TPM 's de nuestra cantidad de nodos a evaluar, se encuentran TPM' s para 15 nodos, 20 nodos, etc.

Veamos un ejemplo para `N10A.csv` que se encuentra en nuestro proyecto:

```
0.585614,0.523173,0.507139,0.566154,0.495793,0.328677,0.346388,0.543257,0.663577,0.386377
0.592450,0.558080,0.344861,0.743432,0.295527,0.553198,0.357183,0.725804,0.651445,0.262984
0.782409,0.226056,0.220426,0.519549,0.216473,0.402498,0.316002,0.261208,0.786124,0.918299
0.841931,0.593135,0.233259,0.613497,0.582071,0.561666,0.272865,0.227723,0.404463,0.635959
0.248847,0.414205,0.639918,0.638686,0.270825,0.225265,0.291313,0.703258,0.714727,0.246295
0.509435,0.551847,0.454361,0.580783,0.230716,0.551554,0.678389,0.203333,0.663092,0.738780
0.675804,0.598754,0.685517,0.297073,0.352824,0.500537,0.656734,0.401731,0.467348,0.754139
0.540523,0.673989,0.646360,0.287590,0.674495,0.460142,0.628712,0.443250,0.843769,0.600950
0.413113,0.520361,0.356299,0.443517,0.568346,0.265696,0.544500,0.897774,0.431773,0.216287
0.593634,0.643619,0.478236,0.788429,0.513233,0.443654,0.790070,0.476463,0.217056,0.779738
0.457305,0.200357,0.638039,0.486348,0.256810,0.233771,0.390352,0.353717,0.570708,0.615064
0.471151,0.280686,0.303838,0.415656,0.223367,0.147886,0.564189,0.292675,0.630066,0.522646
```

Vemos claramente una matriz de donde se observan las probabilidades que nos muestran cómo pasar de un estado a otro.

Ahora pasemos con el archivo de `creation.py`

En `creation` tenemos varios métodos esenciales, porque en este archivo es donde se pueden realizar las muestras de los TPM' s que se utilizan en los sistemas para realizar ese proceso de partición, mediante el uso de estas probabilidades para pasar de un estado al otro.

```
def __init__(self, N: int):
    self.N = N
    self.num_states = 2**N
```

En esta primera función se establece primero como van a funcionar las matrices, que va a ser $2^n * n$ para más estabilidad y optimización a la hora de hacer las evaluaciones, en este `init` también nos ayuda a calcular cuanto espacio ocupara nuestro tpm en `.csv` para los

nodos que le pongamos, nos dará el peso en Gigabytes, por ejemplo con uno de 25 nodos se puede ir hasta las 10 GB.

```
self.states = np.random.uniform(0.2, 0.8, size=(self.num_states, N)).astype(np.float32)
```

Aquí generamos los estados con números aleatorios entre 0.2 y 0.8 para las probabilidades que nos simboliza pasar de un estado a otro, además de unos métodos que determinan las dependencias entre las probabilidades y sus correlaciones.

Se informa después cuánto tiempo se demoró generando el .csv, para ya finalmente pasar a las siguientes funciones que se aplican para la creación de estos TPM's que serían:

```
def marginalize(self, dimension: int) -> np.ndarray:
```

Esta función busca extraer un vector o arreglo de un nodo específico ignorando el resto, simulando la operación matemática de marginalización elemental en el hipercubo, para poder lograr ese $2^n * n$.

```
def save_to_csv(self, filename: str = None):
    target_dir = os.path.join('GeoMIP', 'data', 'samples')
    os.makedirs(target_dir, exist_ok=True)
```

Después guardamos el archivo en nuestra carpeta de samples en formato .csv con una condición y es que dependiendo la cantidad de nodos, si encuentra uno que sean los mismos nodos creará un archivo con una letra acompañante diferente, es decir, si encuentra un N15A, creará un N15B.

```
CHUNK_ROWS = 65_536
    with open(filepath, 'w') as f:
        for start in range(0, self.num_states, CHUNK_ROWS):
            end = min(start + CHUNK_ROWS, self.num_states)
            np.savetxt(f, self.states[start:end],
delimiter=',', fmt='%.6f')
            pct = end / self.num_states * 100
            print(f' [{pct:5.1f}%]
{end:,}/{self.num_states:,} filas escritas', end='\r')
            print()
```

Aquí se usa una limitación para poder generar los TPM's que tienen una cantidad de nodos elevadas, esto se le llama CHUNK el método divide el arreglo en fragmentos de almacenamiento contiguo de 65,536 filas para evitar colgar la creación por enviarlo de forma completa, al igual del uso de punto flotante para optimizar mucho mejor nuestro algoritmo y los datos que utiliza.

Por último se almacena esa .csv y tenemos la función `Main` final que nos pedirá la cantidad de nodos que necesitamos generar y el sistema internamente generará ese TPM de esa cantidad solicitada.

A continuación un ejemplo de consola de cómo lo termina generando.

```
Ingrese el número de variables (N) para el sistema: 12
Generando sistema con N=12...

Tamaño estimado: 0.000183 GB
Tiempo estimado: 0.0 segundos (0.0 minutos)
Generando estados (probabilidades correlacionadas)...
Generación completada en 0.39 segundos

Guardando estados en GeoMIP\data\samples\N12A.csv...
[100.0%] 4,096/4,096 filas escritas
Archivo guardado: 0.000416 GB
Tiempo de guardado: 0.02 segundos
```

Vemos que genera correctamente los estados y los almacena en nuestra carpeta de `samples`.

Ya entendiendo cómo se crean los TPM 's que utiliza el sistema vamos a la carpeta de `results` donde se almacenará los resultados en forma de excel es decir .xlsx donde guardará los tiempos y las pérdidas de los resultados, en el punto 2.7 hablaré con mayor profundidad.

Continuando pasemos ahora a las enumeraciones que se usarán para el proyecto, que son unas constantes que almacenarán los nombres de funciones, palabras claves, para facilitar el llamado en los otros archivos, estas constantes se encuentran en la carpeta `constants` donde hay 3 archivos fundamentales.

`base.py`:

```
INFTY_POS: float = float("inf")
INFTY_NEG: float = float("-inf")

INT_ZERO: int = int(0)
INT_ONE: int = int(1)

FLOAT_ONE: float = float(INT_ONE)
FLOAT_ZERO: float = float(INT_ZERO)

BASE_TWO: int = INT_ONE + INT_ONE

ABC_LEN: int = 26
LAST_IDX: int = -INT_ONE
```

```
ROWS_IDX = ACTUAL = INT_ZERO
COLS_IDX = EFECTO = INT_ONE
```

Encontramos constantes como los infinitos, tanto positivos como negativos, números enteros, números en flotantes, constantes que indican los índices de los arreglos, delimitadores, algunas notaciones, los tipos de estados, configuraciones de entorno y pruebas como .csv o html.

error.py:

```
ERROR_INCOMPATIBLE_SIZES: str = "El estado inicial tiene una  
dimensión diferente con las condiciones, alcance o mecanismo."
```

En el archivo `error.py` encontramos un error descrito por tener una incompatibilidad por las dimensiones, es decir las dimensiones de los conjuntos del estado inicial, las condiciones de fondo, el alcance y el mecanismo deben ser perfectamente consistentes entre sí.

models.py:

```
# System
DUMMY_EMD: int = -1
DUMMY_ARR: list[int] = [-1]
ERROR_PARTITION: str = "No hay suficientes elementos para  
particionar.\n"
DUMMY_PARTITION: str = "NO-PARTITION\n"

# SIA (System Irreducibility Analysis)
SIA_LABEL: str = "sia"
SIA_PREPARATION_TAG: str = f"{SIA_LABEL}_preparation"

# Pyphi
PYPHI_LABEL: str = "Pyphi"
PYPHI_STRATEGY_TAG: str = f"{PYPHI_LABEL}_strategy"
PYPHI_ANALYSIS_TAG: str = f"{PYPHI_LABEL}_analysis"

# Geometric
GEOMETRIC_LABEL: str = "Geometric"
GEOMETRIC_STRATEGY_TAG: str = f"{GEOMETRIC_LABEL}_strategy"
GEOMETRIC_ANALYSIS_TAG: str = f"{GEOMETRIC_LABEL}_analysis"

# K-Partition (búsqueda exhaustiva k=2..5)
KPARTITION_LABEL: str = "KPartition"
KPARTITION_STRATEGY_TAG: str = f"{KPARTITION_LABEL}_strategy"
KPARTITION_ANALYSIS_TAG: str = f"{KPARTITION_LABEL}_analysis"
```

Aquí lo que se busca definir es varios errores o condiciones que se pueden presentar a la hora de evaluar el sistema planteado, por ejemplo si no se puede realizar una partición

porque no se encuentra la cantidad de nodos correspondiente, también pone en constantes el uso de determinada estrategia, si es la geométrica , que tipo de partición se está haciendo.

Pasamos ahora con la definición de la carpeta `video` que busca como podemos visualizar la técnica del hipercubo para la selección de las mejores particiones en base a los mejores cambios, aquí vemos una cantidad de varias versiones pero nos centraremos en la última de ellas que es `hyper-v8.py`.

Hyper-v8 es un visualizador del hipercubo utilizado en el proyecto para lograr las particiones, este se divide en varios métodos, donde tenemos el constructor.

```
class NCubeVisualizer(ThreeDScene):  
    def construct(self)
```

El código está construido en manin, que es una librería de python que nos permite crear animaciones matemáticas, la idea es intentar visualizar un hipercubo de 4 dimensiones, este constructor es el inicializador de este visualizador, se configura la cámara para observar en 3D, crea los arreglos utilizados por los hipercubos, nos ayuda a generar todas las coordenadas binarias posibles en formato (0,0,1,0).

```
def visualize_dimension_reduction(self, data_4d)
```

La idea con esta función del visualizador es realizar una reducción dimensional del mismo, empieza promediando la cuarta dimensión y así sucesivamente para ir reduciendo la dimensión hasta 0 para obtener un único número, y guarda todos esos estados que se van generando, va indicando qué eje se va a eliminando, para después dibujarlo.

```
def create_ncube(self, data, include_labels=True)
```

Esta función intenta construir el objeto de manera visual, primero determina la dimensión del ncubo que acaba de recibir, contiene un caso especial donde genera una esfera si no hay dimensiones encontradas, genera los grupos de los vértices, las aristas y las etiquetas, genera unas coordenadas y obtiene los valores.

```
def get_vertex_position(self, coords, n_dims)
```

Busca convertir las coordenadas binarias a posiciones 3D, dependiendo el hipercubo de cuantas dimensiones usa las dimensiones para crear esas posiciones, simulando el tesseracto.

```
def dim_to_letter(self, dim)
```

Modifica los valores numéricos a letras mediante el uso del Sistema ASCII.

```
def create_color_legend(self)
```

Crea la barra de colores para los valores numéricos.

```
def value_to_color(self, value)
```

Crea los colores para los valores numéricos, los transforma en colores dependiendo del valor dado será un color diferente.

```
class DimensionalReduction(NCubeVisualizer)
```

Esta hereda la clase original para hacer la ejecución de la reducción dimensional, ejecuta la cámara para hacer su configuración, genera el tesseracto y hace el llamado.

Habiendo explicado el visualizador del cubo vamos ahora a la base del proyecto y vamos a la carpeta de `base` donde encontraremos dos archivos `application.py` y `sia.py`. Estos dos cumplen determinadas características que veremos a continuación.

`application.py`:

```
def __init__(self) -> None
```

Esta inicializa todos los parámetros por defecto, este constructor utiliza las notaciones donde se menciona si se necesita guardar el byte más significativo primero o el menos, así como también las métricas de distancia y si un nodo se encuentra activo o no.

```
def set_notacion(self, tipo: Notation)
```

Esta nos permite cambiar la notación según sea necesitado, por ejemplo big-endian, little-endian, estas se mencionan más adelante.

```
def set_distancia(self, tipo: MetricDistance)
```

Esta es como el método de la notación de arriba pero especificado para las distancias, ya que hay diferentes tipos incluso como la Manhattan o la Euclidiana.

```
def set_estados_activos(self)
```

Este método funciona para determinar si uno de los nodos está activo, porque si recordamos, en el sistema que utilizamos hacemos un proceso que nos permite saber con cuales nodos se van a contar o hará que ejecutar para obtener la partición mínima

```
def set_estados_inactivos(self)
```

De igual forma que el método anterior este haría el mismo proceso pero funcionando en el caso contrario, sería con los nodos que no se utilizarán dentro de un sistema.

Ahora para el SIA tenemos que

`sia.py`:

```
def __init__(self, gestor: Manager) -> None
```

El System Integrated Analysis nos sirve como el gestor para el subsistema que se le ejecutarán las estrategias planteadas, en este caso estamos hablando de GeoMIP, este constructor utiliza el Logger y el gestor, así como también recibe el subsistema.

```
@abstractmethod
def aplicar_estrategia(self)
```

Aquí se hace el uso del método abstracto que lo que busca es que cada algoritmo implemente su propia estrategia, es decir cada estrategia tiene esta función con lo que hace respectivamente.

```
def sia_cargar_tpm(self) -> np.ndarray
```

Esta función busca cargar la TPM desde un archivo, como estamos usando .csv está configurado para cargar esos archivos y poderlos leer correctamente, y hacer la función del TPM de manera correcta.

```
def sia_preparar_subsistema
```

Esta es la función más importante de esta clase, es recibir el sistema original, aplicar las condiciones adecuadas de fondo, después obtener sistema candidato del original, substraer las variables que esta genera, hace la obtención del subsistema utilizado por el programa y calcula las marginales, verifica también los tamaños, obtiene las dimensiones condicionadas, define también el alcance al que se puede llegar y guarda finalmente el tiempo utilizado para obtener ese subsistema.

```
def chequear_parametros(self, candidato: str, futuro: str,
presente: str)
```

Esta función valida que todos los parámetros cumplan con la longitud adecuada, y asegura que cada valor evaluado sea igual, en el candidato, en el futuro y en el presente del sistema y en caso de que no, lance una excepción.

Después de explicar la base del proyecto, pasamos ahora al core, es decir el grueso donde se ejecuta internamente el sistema para realizar, en la carpeta de `core` se encuentran 3 archivos fundamentales que desglosaremos a continuación que son el ncube, solution y system.

Empezaremos por el `ncube.py` para después seguir con los demás

`ncube.py`:

```
class NCube
```


Esta clase representa la abstracción matemática de cómo las TPM' s las se pueden representar como los n-cubos que se han hablado con anterioridad, donde se puede condicionar, marginalizar y mantener las dimensiones globales con las locales.

```
def __post_init__(self)
```

Este sirve para validar los datos del constructor de esta clase, hace varias verificaciones, verifica primero si la forma coincide con el número de dimensiones que hay, nos ayuda a prevenir varias inconsistencias en caso de errores.

```
def condicionar(  
    self,  
    indices_condicionados: NDArray[np.int8],  
    estado_inicial: NDArray[np.int8],  
    ) -> "NCube"
```

La funcionalidad de condicionar, es primero mirar el estado inicial que se tiene, y se obtienen los índices condicionados, ambos son unos arreglos, esto nos ayuda a representar un tensor. Esto empieza a recorrer las dimensiones condicionadas, después calcula el eje local, esto lo hace porque numpy almacena desde el exterior al interior, selecciona una cara, para generar nuevas dimensiones y crea un nuevo cubo reducido.

```
def marginalizar(self, ejes: NDArray[np.int8]) -> "NCube"
```

Aquí tenemos la función de marginalizar, que la idea de la marginalización es opacar una de los nodos para no tenerlo en cuenta, y poder un cálculo más óptimo, empieza a mirar los ejes y decide cual eliminar, de ahí empieza a verificar la intersección, mira si existen y si no, no hace nada, pasa a mirar el número de dimensiones y mirar los ejes locales. Empieza a mirar las nuevas dimensiones que se generan después de marginalizar, realiza un promedio y nos entrega un tensor se hace una Marginalización uniforme, porque son las que mejor nos funcionan con las TPM' s que son de estado a nodo.

```
def __str__(self) -> str
```

Es una función para imprimir más entendible para el usuario final que va a usar este proyecto, hace una mejor depuración y muestra más claridad al final

Pasamos ahora con la solución, que es la que nos permite visualizar las soluciones que saca el sistema y podamos visualizarlo en consola.

solution.py:

Es la encargada de representar el resultado final del algoritmo SIA. Es básicamente el objeto que encapsula, lo que es la mejor partición mínima, las distribuciones comparadas, el tiempo de ejecución, la estrategia utilizada, nos entrega la respuesta final al problema de IIT.

```
def __init__(  
    self,
```

```

    estrategia: str,
    perdida: float,
    distribucion_subsistema: np.ndarray,
    distribucion_particion: np.ndarray,
    particion: str,
    tiempo_total: float = FLOAT_ZERO,
    hablar: bool = True,
    voz: Optional[str] = None,
    k_particiones: int = 2,
    resultados_por_k: dict | None = None,
) -> None

```

Este constructor como se mencionó arriba, contiene la respuesta final que mostrará el sistema cuando acabe de ejecutarse, aquí tenemos la pérdida que tendrá el subsistema en su mejor forma de partir, la estrategia que se utiliza, el tiempo que se demoró en realizar la solución y cada resultado por cada K que se utilizó.

```
def __obtener_voz_espanol(self, motor: Engine) -> Optional[str]
```

La idea de esta función es entregar voz a los resultados obtenidos, que las voces de un robot nos digan que acaba de sacar el programa, se eligen entre varios y distintos idiomas, la idea es que sea opcional, no es necesario utilizarla.

```
def __anunciar_solucion(self) -> None
```

Aquí hace la configuración de la voz y hace lo necesario para que cuando el programa acabe esta anuncie los resultados mediante la voz del robot. se tienen los parámetros de que tan lento habla, el tono, el volumen entre otras opciones.

```
def __str__(self) -> str
```

Esta es una de las partes más importantes porque esta función nos muestra de una forma clara cómo se imprime en la consola, cada dato que obtuvo y que le envía al usuario final, aquí se miran todos los parámetros que nos entrega la solución final.

```
def formatear_distribucion(
    distribucion: np.ndarray,
    evitar_desbordamiento=True,
)

```

Formatean las distribuciones, limitan el tamaño a 64 para evitar un desbordamiento, pone colores a la consola para mostrar con más claridad que es cada parámetro mostrado, hace el síntesis de la voz, detecta si hizo uso de pyphi marca el tiempo y hace el retorno de la información final.

Este sería un ejemplo de como mostraría la solución:

```
Brute Force fue la estrategia de solución.
```

```
Distancia métrica utilizada:  
EMD_EFECTO
```

```
Notación utilizada:  
LITTLE_ENDIAN
```

```
Distribución marginal del Subsistema:  
[0.25 0.25 0.25 0.25]
```

```
Distribución marginal de la Partición:  
[0.20 0.30 0.25 0.25]
```

```
Mejor Partición (k=2):  
AB | CD
```

```
Pérdida mínima ( $\phi$ )=0.0345
```

```
Tiempos de ejecución:
```

```
Horas: 0.00  
Minutos: 0.3  
Segundos: 18.2345
```

```
def __repr__(self) -> str
```

Es la forma de hacer el llamado a la función del str y siempre se usa para que llame a str y muestre los resultados con ese formato.

Vamos ahora con el archivo de System donde se encarga de realizar las operaciones de condicionamiento, substracción para generación de subsistemas y obtención de las distribuciones marginales para realizar eficientemente el cálculo de la EMD en el Efecto.

System.py:

Esta clase System es una de las clases más importantes del proyecto, porque es la que agrupa todos los n-cubos y permite realizar las operaciones de IIT sobre el sistema completo.

```
def __init__(  
    self,  
    tpm: np.ndarray,  
    estado_inicio: np.ndarray,  
    notacion: str = aplicacion.notacion,  
)
```

Esta función de inicialización recibe el TPM completo del sistema que se va a usar, nos da el estado inicial y la notación que ésta utiliza, ya sea big-endian o little-endian.

Hace una validación donde comprueba que la cantidad de nodos si representa la TPM que corresponde a esa cantidad de nodos, es decir, si por ejemplo tenemos 4 nodos, hay 16 líneas en el TPM representando todos los valores binarios posibles para el cambio de un lado a otro, si no cumple esa validación se lanza un excepción. Guarda los estados y construye los n-cubos.

```
@property
def indices_ncubos(self)
```

Hace el condicionamiento y la sustracción de los n-cubos, si tenemos un arreglo que llega de [0,1,2,3] después de hacer el condicionamiento pasa a [0,2,3] y después de la sustracción pasa a [0,3].

```
@property
def dims_ncubos(self)
```

Aquí se definen qué variables siguen vivas, dependiendo de las dimensiones que haya recibido el n-cubo.

```
def condicionar(self, indices: NDArray[np.int8]) -> "System"
```

La función de condicionar, su principal labor es la de darnos un sistema candidato, empieza validando los índices mediante una intersección, de ahí nos crea el nuevo sistema que se utiliza para las pruebas, estos se reservan en memoria porque ya tenemos los n-cubos, no hay que reconstruirlos de nuevo desde la TPM. se condiciona cada cubo para eliminar completamente los cubos condicionados.

```
def substraer(
    self,
    alcance_dims: NDArray[np.int8],
    mecanismo_dims: NDArray[np.int8],
) -> "System"
```

Con substraer, también se genera un sistema candidato, este recibe el alcance de las dimensiones y el mecanismo de las dimensiones, es decir el t y el $t+1$, valida los futuros que son válidos y subtrae los nodos que dejan de utilizarse, después se marginaliza y nos da el sistema reducido.

```
def bipartir(
    self,
    alcance: NDArray[np.int8],
    mecanismo: NDArray[np.int8],
) -> "System"
```

Es la operación central que hará nuestro sistema, mediante el uso del alcance y el mecanismo, busca la mejor forma de partirse para obtener la pérdida mínima, primero analiza cada cubo para dejar nomas las variables exceptuando las del mecanismo, esto en

caso de que el cubo pertenezca al alcance, si no pertenece se marginaliza y se elimina el mecanismo lo que nos deja la TPM particionada.

```
def particionar_k(
    self,
    partes_purview: list[NDArray[np.int8]],
    partes_mecanismo: list[NDArray[np.int8]],
) -> "System"
```

Aquí generalizamos las biparticiones, no solamente de partir en dos si no escalar a algo más complejo, donde se pueda partir en 3 o más. se crea un mapa que permite entender la forma de partirse según k, y empieza a calcular los mecanismos de los otros y hace el mismo proceso marginalizado.

```
def distribucion_marginal(self)
```

Esta función es importante porque nos permite determinar el cálculo de EMD, empieza reservando un vector, que ahí serán las distribuciones posibles de cómo se puede partir, empieza analizando los cubos y obtener la probabilidad inicial, si el cubo tiene dimensiones crea el estado inicial, de ahí seleccionamos un estado, que sería un sub estado, y obtiene la probabilidad específica del estado inicial.

```
def __str__(self) -> str
```

Permite imprimir, el comportamiento de cómo se van adecuando los n-cubos por medio de las funciones que se utilizan en esta clase, dándole una mejor vista general al usuario que ejecute el proyecto.

Para ir finalizando los modelos nos queda una última carpeta llamada `enums` vamos utilizar dos archivos donde nombramos las distancias y las notaciones que se van a utilizar dentro del proyecto, son notaciones binarias.

distance.py:

```
class MetricDistance(Enum):
    """La clase notaciones recopila diferentes notaciones
    binarias. Al definir el tipo se accede por `.value`."""

    EMD_EFECTO = "emd-effect"
    EMD_CAUSA = "emd-cause"
    MANHATTAN = "distancia-manchattan"
    EUCLIDIANA = "distancia-euclidiana"
```

Aquí se observan las principales distancias que se utilizan para las métricas dentro del proyecto, se observan 2 tipos de Distancia EMD que serían la de causa y efecto, la distancia Manhattan y Euclidiana, la euclidiana sirve para medir la separación geométrica más corta y directa entre dos puntos en un espacio continua, la de manhattan sirve para medir la distancia sumando las diferencias absolutas de cada dimensión.

notation.py:

```
class Notation(Enum):  
    """La clase de notaciones recopila diferentes notaciones  
    binarias. Al definir el tipo se accede por `.value`."""  
  
    LIL_ENDIAN = "little-endian"  
    BIG_ENDIAN = "big-endian"  
    GRAY_CODE = "gray-code"  
    SIGN_MAGNITUDE = "sign-magnitude"  
    TWOS_COMPLEMENT = "two's-complement"
```

Aquí definimos las notaciones de formatos de números binarios, en el primero de little-endian nos referimos a guardar el byte menos significativo primero en memoria, después el big-endian es guardar el byte más significativo primero en la memoria, el sign-magnitude nos dice cuando el primer bit indica si es un positivo o un negativo y el two's-complement es el complemento a dos de un número en binario, por último el código grey es un sistema binario donde dos números consecutivos difieren solo en un bit.

Pasamos ahora a otra sección que es la de los `middlewares`, en esta carpeta encontramos dos archivos, `profile.py` y `slogger.py`. El `profile` funciona como alguien que responde preguntas como ¿Qué función tarda más tiempo?, ¿Dónde están los cuellos de botella?, ¿Cuántas veces se llama una función?, ¿Qué porcentaje del tiempo total consume cada parte del algoritmo?, el `slogger`, en cambio responde preguntas como Este módulo responde preguntas como: ¿Qué está haciendo el algoritmo en cada paso?, ¿Cuándo ocurrió un error?, ¿Qué parámetros se utilizaron?, ¿Qué pasó antes de que fallara? Este `middleware` funciona como un conector de varios de los componentes que se usan en el proyecto y nos da información fundamental para entender con mayor claridad los resultados de las ejecuciones.

Slogger.py:

```
class ColorFormatter(logging.Formatter):  
  
    def __init__(self, *args, **kwargs)
```

Inicializa el colorama, para los colores.

```
    def format(self, record: logging.LogRecord) -> str
```

Sobreescribe el formato original, es decir, obtiene el color, guarda el nombre original, le añade color, y llama al formateador padre.

```
class SafeLogger:
```

Esta es la clase principal, su función es guardar los registros sin romperse, guarda incluso objetos que no tendrían que estar, todo con la idea de obtener toda la información posible.

```
def __init__(self, name: str)
```

Este constructor es el nombre que se le da al logger que va a hacer, es decir el nombre a la clase que se está haciendo el log.

```
def _safe_str(self, obj: Any) -> str
```

Nos permite convertir cualquier objeto a string, esto con la idea de evitar errores de codificación en UTF-8, si hay un error, se envía la información de que no se pudo representar pero sin tirar la aplicación

```
def _safe_format(self, *args, **kwargs) -> str
```

Combina todos los parámetros para que se puedan unir en oraciones sin problemas, si por ejemplo tenemos que unir los valores de partición mínima, con el phi ϕ se hace sin problemas.

```
def __setup_logger(self, name: str) -> logging.Logger
```

Es el creador base de esta clase, crea la carpeta donde se ubicaran los logs, si no existe la carpeta la crea y si no es el caso, la ubica donde corresponde, y guarda los logs. siempre sobrescribe entonces es bueno para mirar las últimas ejecuciones que suele suceder.

```
def set_log(self, level: int, *args, **kwargs) -> None
```

```
def debug(self, *args, **kwargs) -> None:
    """Log a nivel DEBUG."""
    self.set_log(logging.DEBUG, *args, **kwargs)

def info(self, *args, **kwargs) -> None:
    """Log a nivel INFO."""
    self.set_log(logging.INFO, *args, **kwargs)

def warn(self, *args, **kwargs) -> None:
    """Log a nivel WARNING."""
    self.set_log(logging.WARNING, *args, **kwargs)

def error(self, *args, **kwargs) -> None:
    """Log a nivel ERROR."""
    self.set_log(logging.ERROR, *args, **kwargs)

def critic(self, *args, **kwargs) -> None:
    """Log a nivel CRITICAL."""
    self.set_log(logging.CRITICAL, *args, **kwargs)

def fatal(self, *args, **kwargs) -> None:
    """Log a nivel FATAL."""
```

```
self.set_log(logging.FATAL, *args, **kwargs)
```

Corresponden a logs para determinadas situaciones, si por ejemplo hubo algún error, algún aviso o información, algún debugging, y el método genérico del log.

```
def get_logger(name: str) -> SafeLogger
```

Nos permite tener una instancia del logger, es una función conveniente para facilidades.

```
def log_execution(logger: SafeLogger)
```

Este funciona como un decorador, permite hacer la ejecución del log, registra errores, lanza errores y excepciones.

Vamos ahora con el segundo middleware que es profile.

Profile.py:

```
class ProfilingManage:
```

```
    def __init__(
        self,
        habilitado: bool = aplicacion.profiler_habilitado,
        dir_salida: Path = Path(PROFILING_PATH),
        intervalo: float = 0.01,
    )
```

Aquí controlamos si se encuentra habilitado el profiling para realizarlo, determina la ubicación donde se harán los profilings, toma un intervalo que toma muestras, estas muestras si son pequeñas, tendrán mucha precisión pero mucho overhead, pero entre mas grande, menos overhead pero a cambio de menor precisión.

```
def _setup_directories(self) -> None
```

Este crea las carpetas solo si está activo, verifica el directorio, y almacena el profiling en esa ubicación.

```
def start_session(self, session_name: str) -> None
```

El start_session nos permite agrupar perfiles, es decir crear una carpeta por el tipo de estrategia utilizada y poner los datos correspondientes con esa estrategia. se guarda esa sesión actual.

```
def get_output_path(self, name: str, format: str) -> Path
```

Esta es la que nos permite construir los nombres para los archivos que se van a guardar, por ejemplo particion.html.


```
class ProfilerContext:

    def __init__(
        self,
        manager: ProfilingManager,
        name: str,
        context: dict,
    )
```

Es un manejador de contexto, entra al profiling y al final deja un reporte de lo que hizo. Este constructor almacena el nombre del reporte y la información adicional, es decir el contexto.

```
def __enter__(self)
```

Se ejecuta al comenzar un profiling, toma el tiempo de alta precisión, e inicializa el PyInstrument, se retorna a sí mismo.

```
def __exit__(self, exc_type, exc_val, exc_tb)
```

Se ejecuta cuando acaba de hacer el profiling, lo hace de forma automática, si está deshabilitado no hace nada. detiene el profiler y almacena el tiempo que se demoró, hace el proceso de guardado en html.

```
def profile(name: Optional[str] = None, context: Optional[dict] =
None) -> Callable
```

Esto crea un decorador configurable, este recibe los parámetros del decorador que permite medir automáticamente el rendimiento de cualquier función sin modificar su código original.

```
def decorator(func: Callable) -> Callable
```

Este recibe la función que se quiere perfilar, esta contiene wrapper que lo que hace es ejecutar cuando se llama a la función: verifica si el profiling está habilitado, inicia el ProfilerContext, éste comienza a medir tiempos y capturar información con PyInstrument, ejecuta la función original y, al finalizar, detiene el profiler y genera un reporte HTML con los resultados del análisis de rendimiento.

Ya nos encontramos en las funciones que contiene nuestro proyecto, estas se ubican en la carpeta de funciones `funcs` aquí encontraremos componentes importantes que le dan vida al proyecto.

Empezamos con `base.py`

El propósito de este archivo es resolver de manera altamente optimizada las operaciones algebraicas, de formateo, indexación de memoria y cálculo de distancias probabilísticas que las estrategias de búsqueda necesitan consumir iterativamente.

```
def get_labels(n: int) -> tuple[str, ...]
```

Esta función lo que busca es obtener los índices numéricos de los nodos para volverlos letras, busca es darnos según la cantidad de nodos que le dimos, obtener los nodos en el abecedario, por ejemplo, 4 nodos sería (A,B,C,D)

```
def literales(remaining_vars: NDArray[np.int8], lower: bool = False)
```

Permite transformar un arreglo numérico de índices de nodos en una cadena de texto estilizada. Es la encargada de traducir representaciones de memoria, la idea es devolver un string formateado para, con las letras de forma adecuadas, así sean mayúsculas o minúsculas.

```
def seleccionar_metrica(distancia_usada: str)
```

Permite al sistema intercambiar dinámicamente el algoritmo matemático con el cual se evaluará la divergencia de los datos en base a la configuración elegida por el usuario, identificando el tipo de distancia usada, ya sea la causal o de efecto.

```
def emd_efecto(u: NDArray[np.float32], v: NDArray[np.float32]) -> float
```

Diseñada específicamente para la evaluación del repertorio de efectos bajo la premisa de variables condicionalmente independientes, es decir, devuelve un punto flotante que representa la distancia absoluta acumulada.

```
def emd_causal(u: NDArray[np.float64], v: NDArray[np.float64]) -> float
```

Esta busca encontrar el cálculo del EMD pero para las EMD Causales, se utiliza cuando el sistema quiere evaluar la pérdida de la información en nodos donde no hay una dependencia directa entre ellas.

```
def hamming_distance(a: int, b: int) -> int
```

Nos permite obtener la distancia geométrica de dos puntos de un hipercubo, mediante el contador de bits, con los cambios que se tendrían que hacer para llegar al resultado original. cuantos nodos cambian de un punto a otro.

```
def reindexar(N: int)
```

Buscamos darle soporte a la notación según sea la elegida, ya sea una little-endian, o un big-endian.

```
def seleccionar_subestado(subestado)
```

Se hace una posible in-deducción por acceso inverso.

```
def lil_endian(n: int) -> np.ndarray
```

Reorganiza el orden de lectura de los ejes binarios para asegurar la compatibilidad matemática con los formatos de datos heredados o configuraciones de PyPhi. Debido a la explosión combinatoria se busca encontrar el procesamiento por bloques contiguos (Chunking), esto para agruparlos y obtener bloques más avanzados.

```
def get_restricted_combinations(binary_str: str) -> tuple[list[str], list[str]]
```

Genera conjuntos de subestados binarios condicionados de manera jerárquica. Se encarga de filtrar el espacio de búsqueda restringiendo que los bloques dependientes solo muten o posean activaciones, esto para el padre que está activo y tener mejor precisión.

```
def generate_combinations(A: str) -> list[tuple[str, str, str]]
```

Es donde generamos las combinaciones de nuestro sistema que irá almacenando y determinando a raíz de cuántos nodos se tengan, una tupla que almacenará todos los resultados del hipercubo.

```
def dec2bin(decimal: int, width: int) -> str
```

Formatea un número entero a su representación binaria, en formato de texto mediante su longitud.

```
def estados_binarios(n: int)
```

Genera de forma secuencial todo el universo de estados binarios posibles para un espacio de dimensión n .

Vamos ahora con el siguiente archivo que sería `excel_pruebas2026.py`

Este archivo busca llenar los resultados que el programa va obteniendo en el excel que le pongamos, así automatizando este proceso de los resultados.

```
def _celda_valor(df: pd.DataFrame, fila: int, col: int) -> str
```

Esta función asegura que si dentro del excel, si encuentra que hay una celda con algún error o se encuentre vacía, no dará un error, sino que devolver la cadena de texto vacía.

```
def letras_a_binario(texto: str, n_bits: int) -> str
```

Como en los excels de pruebas que han enviado para el proyecto hacen uso de las letras para mostrar los nodos, esta función se encarga de traducir esas letras a los 1 y 0 para que nuestro programa lo pueda entender y sacar las soluciones a sus sistemas.

```
def sistema_candidato_a_condicion(sistema: str, candidato: str, n_bits: int) -> str
```

Esta función define los nodos que vamos a evaluar activamente en la partición que se hace en el momento, y cuáles son los que no se utilizarán ya que se quedan en 0, es separar los que están partiendo con los que se quedan afuera de la ecuación.

```
def parsear_nombre_hoja(nombre: str) -> tuple[int, str] | None
```

Intenta encontrar en los archivos de tipo excel, nombres semejantes a los que necesita el proyecto, para poder procesarlos, si se sale de esos estándares no evaluará esa hoja con las particiones a encontrar. Si encuentra que la hoja es válida obtiene los valores de la cantidad de nodos para saber de cuántos nodos se está evaluando en ese momento.

```
def leer_hoja_pruebas(ruta: Path, nombre_hoja: str) -> list[Prueba2026]
```

Es que hace la organización de la información, entra al archivo, y empieza a identificar los valores, que necesita, el estado inicial, el sistema que se está utilizando, y empieza a evaluar de ahí para abajo todos los casos donde se piden hallar las particiones los mecanismos y alcances que nos están pidiendo.

```
def escribir_resultado_fila(
    ws,
    fila: int,
    resultados_por_k: dict,
    mejor_k: int | None,
    mejor_perdida: float | None,
) -> None
```

Aquí empieza a escribir cuando haya evaluado todos los casos, con los distintos K, cuál fue la partición de menor pérdida y con cual cantidad de particiones se logró.

```
def guardar_workbook(ruta_entrada: Path, ruta_salida: Path) -> None
```

Se genera un archivo nuevo donde se almacenarán estos resultados de ese archivo, lo cual nos da una mejor practicidad y evita modificar el original.

format.py:

Este archivo busca principalmente las respuestas que da la máquina, que son un montón de listas, direcciones, etc. sean formateadas a un tipo de lectura entendible para el usuario.

```
def fmt_biparticion(
    parte_uno: list[tuple[int, ...], tuple[int, ...]],
    parte_dos: list[tuple[int, ...], tuple[int, ...]],
) -> str
```

Este nos permite obtener las particiones que hace el sistema por medio de las letras y el uso de este | para mostrar como se partio y que resultado da esa forma de cortarlo, un

ejemplo que se puede realizar para mostrar eso sería BC|D y pone un símbolo de vacío ∅ en caso de que se use en una de las particiones.

```
def fmt_biparte_q(
    prim: list[tuple[int, int]],
    dual: list[tuple[int, int]],
    to_sort: bool = True,
) -> str
```

Funciona como un formateador para pegar un bloque arriba y otro abajo, asegurándose de que las líneas de arriba sean el futuro y las de abajo sean el presente.

```
def partes_a_clave(
    partes_purview: list,
    partes_mecanismo: list,
) -> tuple[tuple[tuple[int, int], ...], ...]
```

Es una función que busca optimizar las formas de como se parte cuando el programa hace su trabajo, convierte los cortes en una clave inmutable, y usa esas claves para guardar los resultados en memoria caché y así obtener la respuesta más rápido ya que evitará caer en ese mismo corte más adelante.

```
def fmt_k_particion(partes: list[list[tuple[int, int]]]) -> str
```

Este nos permite obtener las particiones que hace el sistema por medio de las letras y el uso de este | para mostrar como se partio y que resultado da esa forma de cortarlo, aplicado a cuando hay más de un k=2, es decir se encarga de hacer exactamente lo mismo que la otra función pero para k mayores un ejemplo con k=4 puede ser A|B|C|DEF|G

```
def fmt_parte_q(parte: list[tuple[int, int]], to_sort: bool = True) -> tuple[str, str]
```

Nos ayuda a construir los conjuntos de nodos por alcance y mecanismo, y los divide en los dos grupos de como fueron obtenidos un ejemplo sería y los ubica de forma ordenada.

```
| AB | > Alcance
| ad | > Mecanismo
```

partitions.py:

Este archivo busca hacer el trabajo pesado o el trabajo real que necesitamos lograr, es buscar la forma de como dividir todos los nodos que se están evaluando, tanto en el presente como en el futuro, es encontrar todas las formas de dividirlos para las k-particiones.

```
def _asignacion_a_partes(
    indices: NDArray[np.int8],
    asignacion: tuple[int, ...],
```

```

        k: int,
    ) -> list[NDArray[np.int8]]

```

Nos permite tener los grupos y cómo se van a ir combinando los nodos que estamos evaluando, mediante los grupos se ubican los nodos y las formas de cómo estos se están partiendo.

```

def es_particion_trivial(
    partes_purview: list[NDArray[np.int8]],
    partes_mecanismo: list[NDArray[np.int8]],
) -> bool

```

Este es un filtro que se está haciendo, una partición trivial funciona como una especie de corte falso, es decir deje todo igual pero lo corta con un vacío, eso no tiene sentido y esta función evita que esto ocurra. Si llega a encontrar una partición de este estilo la descarta inmediatamente.

```

def enumerar_k_particiones(
    dims_mecanismo: NDArray[np.int8],
    indices_purview: NDArray[np.int8],
    k: int,
) -> Generator[tuple[list[NDArray[np.int8]],
list[NDArray[np.int8]]], None, None]

```

Este nos permite calcular la cantidad de combinaciones que se pueden hacer dependiendo el k que esté evaluando y la cantidad de nodos del sistema, entonces si por ejemplo son 30 nodos este se encarga de la cantidad de particiones que se pueden generar para las 30 variables en cada k que se esté evaluando.

```

def enumerar_biparticiones_binarias(
    dims_mecanismo: NDArray[np.int8],
    indices_purview: NDArray[np.int8],
) -> Generator[tuple[list[NDArray[np.int8]],
list[NDArray[np.int8]]], None, None]

```

Es una función que pretende optimizar cuando se trate de obtener k=2 en los nodos, ya que al ser la forma de partir más común y repetitivo, entonces hace un proceso mediante un atajo usando los unos y ceros para evitar evaluar los que no se están utilizando.

System.py:

Este nos permite generar la estrategia para empezar a evaluar los sistemas del proyecto, aquí es donde se proponen los subsistemas, los candidatos y las combinaciones de forma más inteligente y óptima para nosotros.

```

def generar_candidatos(n_vars: int)

```

La idea con esta función es determinar los candidatos que vamos a utilizar para evaluar, lo que hace es obtener pequeños grupos donde empezaremos a evaluar, constantemente hasta tener el sistema complejo, ahorrando trabajo para la máquina y obteniendo mejores resultados para obtención de la partición.

```
def generar_subsistemas(vars: tuple[int])
```

Nos permite medir la información que se necesita evaluar en un mecanismo en el presente y ver su efecto en el futuro. Esta función agarra un grupo de nodos y calcula todas las parejas posibles mediante un producto cartesiano. Así logramos que el sistema sepa exactamente qué combinaciones de presente y futuro debe poner a prueba.

```
def generar_particiones(
    m: int,
    n: int,
    *,
    as_matrix: bool = False,
    as_generator: bool = True,
) -> Union[Generator[Tuple[np.ndarray, np.ndarray], None, None],
np.ndarray]
```

Esta función está optimizada para que genere cortes de forma más óptima mediante el uso de 1 y ceros para las biparticiones. Esta genera millones de cortes en un instante gracias a la función de `as_generator`, esto ayuda a evitar el colapso del computador por memoria RAM.

```
def biparticiones(
    alcances: np.ndarray,
    mecanismos: np.ndarray,
    total=None,
)
```

Toma todos los subconjuntos del futuro lo que serían los alcances y todos los subconjuntos del presente, es decir los mecanismos y los combina. este usa el `isslice` para evitar dar cortes que se deberían descartar como las primeras combinaciones.

```
def subconjuntos(arr: np.ndarray)
```

Este principalmente toma una lista de nodos y devuelve todas las combinaciones posibles que se pueden armar con ellos.

Ya vamos finalizando con la explicación de nuestro algoritmo de GeoMIP, hasta el momento se ha hablado de lo que es la descripción del problema en cuestión como este funciona y cómo debería hacer su trabajo, vamos ahora con las fases finales que es como funciona nuestra estrategia geométrica para hacer uso de todos esos elementos encontrados, como funciona el archivo principal para orquestar todo esto y finalmente la ejecución del proyecto.

Vamos ahora a la carpeta de los controladores (`controllers`) y encontraremos el `manager.py` dentro estara la carpeta de estrategias (`strategies`) encontraremos la estrategia principal geometrica, denominada `geometric.py` ahora si empezamos por el manager que tenemos instaurado en nuestro proyecto.

`manager.py`:

La idea del manager es que nos sirva como el constructor de entornos del proyecto, su función es ubicar los TPM' s que tenemos generados y por otra parte entender de que si no encuentra una red a la cual evaluar el por su cuenta tiene una función que le permitirá crearla de ser necesario.

```
class Manager:

def __post_init__(self) -> None
```

Funciona como un rastreador automático, ya que busca la ruta donde tenemos las muestras del TPM' s para que las use, sin importar los demás detalles, esta recibe también el estado inicial que se quiere analizar para que ahí se haga esa búsqueda.

```
@property
def tpm_filename(self) -> Path
```

Este ayuda exactamente que archivo debe buscar y dependiendo lo que diga el archivo de excel saber exactamente qué tpm' s terminar usando

```
@property
def output_dir(self) -> Path
```

Le dice al sistema en qué carpeta exacta debe guardar los reportes de auditoría y análisis una vez que el algoritmo termine de ejecutarse.

```
def generar_red(
    self,
    dimensiones: int,
    datos_discretos: bool = True,
    pagina: str | None = None,
    auto_confirm: bool = False,
) -> str | None
```

Esta se encarga de la creación y del nombramiento del archivo en .csv, si por ejemplo queremos generar un red de 5 nodos, N5, este inmediatamente asocia una letra a ese nombre N5A, Si queremos ejecutar otra red del mismo tamaño la función se encarga de continuar el abecedario N5B.csv.

```
def _escribir_tpm(
    self,
    dimensiones: int,
```



```

        suffix: str,
        variant_number: int,
        datos_discretos: bool,
        auto_confirm: bool,
    ) -> str | None

```

Funciona como un fabricante de matrices, en caso de no tener la red disponible para realizar el proceso, se ejecuta esta función, pero hace unos pasos determinados, primero nos calcula el impacto, es decir, mira cuánta memoria RAM va a ocupar la red y cuanto va a pesar el archivo. Después genera datos realistas, mediante el uso de una semilla, hace una simulación de las conexiones, osea, los vecinos simulando comportamientos reales, y el guardado mediante el chunking para guardarla por bloques, y evitar colgar el computador.

Ahora vamos con la estrategia planteada por el proyecto GeoMIP.

geometric.py:

Aquí implementamos la estrategia geométrica, evitando recorrer absolutamente todo como la Fuerza Bruta haría, para obtener un mejor rendimiento y optimización.

```

def __init__(self, gestor: Manager)

```

Este constructor, cuando se llega a él ejecuta un profiling para medir los tiempos de ejecución del programa, activa una forma segura de imprimir en la consola e inicializa unos diccionarios para cálculos pesados y guardar la geometría del hipercubo.

```

def aplicar_estrategia(
    self,
    condicion: str,
    alcance: str,
    mecanismo: str,
    tpm: np.ndarray,
) -> Solution

```

Este es el orquestador principal de esta clase, es el que llama a todos los elementos de afuera para realizar las pruebas. Primero recibe la matriz TPM junto con el mecanismo y el alcance que se quiere probar, estandariza los n-cubos, define los nodos del presente y del futuro, finalmente llama al motor de búsqueda find_mip para que encuentre el mejor corte.

```

def find_mip(self) -> tuple[int, list, float, np.ndarray]

```

El objetivo de esta función es obtener los cortes para el sistema con cada k solicitado, para registrar el mejor corte en cada finalización de k, primero construye una tabla con los costos geométricos del sistema, manda a identificar cuales son los k para cortar en base al hipercubo y evitar cálculos de más, revisa constantemente la memoria de particiones para tener unos guardados para sacar la respuesta inmediatamente, finalmente genera un reporte final por cada k evaluado cada vez que termina.

```
def _construir_tabla_costos(self) -> None
```

Encargada de crear el mapa de las distancias para saber cuales nodos están más conectados o desconectados con otros nodos, ya que necesitamos saber cuánto cuesta pasar de un estado de red a otro.

```
def identificar_candidatos_k(
    self, k: int
) -> List[Tuple[List[List[int]], List[List[int]]]]
```

La idea es encontrar los cortes más viables en k evitando tener que evaluar todos los casos o combinaciones como lo haría con la fuerza bruta. Esto funciona así, si encontramos una bipartición se usa un método directo que se basa en el costo del mapa. Se selecciona unos estados estratégicos en el espacio y agrupa los nodos, se hace divisiones recursivas para generar subgrupos de forma geométrica, finalmente junta nodos que no dependen entre ellos para abaratar los costos y evitar romper enlaces que afectaron a la respuesta final.

```
def _candidatos_biparticion(self) -> List[Tuple[List[int],
List[int]]]
```

Busca los cortes a la mitad de forma eficiente, mediante la búsqueda del complementario entre los estados. Suma la tabla de costos de las transiciones para averiguar cuales terminan siendo las más económicas de aislar. Luego mediante el uso de la distancia de Hamming.

```
def _biparticion_a_k_partes(
    self, presentes: List[int], futuros: List[int]
) -> Tuple[List[List[int]], List[List[int]]]
```

Simplemente toma una propuesta de corte de dos partes izquierda y derecha y la procesa a través del formateador general para asegurarse de que sea compatible con la estructura de bloques que requiere el resto del programa.

```
def _expandir_a_k_partes(
    self,
    presentes: List[int],
    futuros: List[int],
    k: int
) -> Tuple[List[List[int]], List[List[int]]]
```

Funciona como un divisor en cascada, ya que toma un corte básico como una propuesta es decir una bipartición, si se llega a necesitar un corte de un k mayor, busca cual es el bloque más adecuado con el mayor de nodos acumulados. A ese bloque lo pasamos por ese ranking de costos y lo cortamos a la mitad de manera balanceada y repite hasta el k que se necesita.

```
def _seleccionar_ancoras(self, k: int) -> List[List[int]]
```

El objetivo aquí es encontrar las capitales del sistema, unos estados que se les denomina como anclas. Para dividir una red en partes, primero busca k estados dentro del hipercubo cumplan dos condiciones, que tengan un costo de transición muy bajo y que estén bastante alejados entre sí, mediante el uso de hamming que haya que cambiar muchos bits para llegar.

```
def _asignar_por_ancoras(
    self, anclas: List[List[int]], k: int
) -> Tuple[List[List[int]], List[List[int]]]
```

Esta función toma cada nodo del presente y del futuro y calcula a qué capital le sale más barato conectarse mediante las capitales que nos dio el método anterior.

```
def _division_rekursiva(self, k: int) -> Tuple[List[List[int]],
List[List[int]]] | None
```

Es la aplicación del algoritmo de divide y vencerás, aplicando el corte geométrico adecuado para lograr ese óptimo en un subgrupo encontrado

```
def _mejor_biparticion_local(
    self, idxs_mec: List[int], idxs_fut: List[int]
) -> Tuple[List[int], List[int]]
```

Aquí se divide el territorio de forma matemática y repitiendo la operación de manera recursiva hasta completar las k divisiones.

```
def _agrupar_por_costo_cero(
    self, k: int
) -> Tuple[List[List[int]], List[List[int]]] | None
```

Toma los costos de transición desde el estado inicial de la red hasta el estado final complementario, es decir el extremo del hipercubo, y ordena las variables del futuro de menor a mayor costo y las va repartiendo en los k bloques usando una operación de residuo matemático.

```
def _particion_valida(self, partes_pur, partes_mec, k)
```

Esta busca asegurar que la partición que se está haciendo sea válida, cumpliendo unos principios necesarios. Que el número de grupos sea igual al K solicitado y que ningún grupo se quede totalmente vacío.

```
def _clave_candidato(partes_pur: List[List[int]], partes_mec:
List[List[int]]) -> tuple
```

Para que el sistema pueda usar la memoria caché y recordar si ya se evaluó un corte antes, necesita algo que permita identificarlo como único. Esta función ordena internamente los números de cada grupo y los convierte en una tupla fija e inmutable.

```
def calcular_costos_nivel(self, estado_final: np.ndarray, nivel)
```

Toma los estados calculados en el nivel anterior y empieza a cambiar un solo bit a la vez en la dirección del estado final. Cada vez que descubre una esquina nueva en el hipercubo, calcula su costo acumulado y la registra en la lista de caminos para que el algoritmo pueda seguir avanzando al siguiente nivel.

```
def calcular_costo(self, estado_inicial: tuple, estado_final: tuple,
ncubos: list[int])
```

Implementa la ecuación de costo geométrico fundamental del proyecto

$$t_X(i,j) = \gamma \cdot (|X[i] - X[j]| + \sum t_X(k,j))$$

```
def hamming(a: List[int], b: List[int]) -> int
```

Es la regla de medir bits. Simplemente compara dos cadenas de unos y ceros posición por posición y cuenta en cuántas de ellas son diferentes.

Ya habiendo explicado nuestra estrategia, es hora de hablar de nuestros orquestadores que hacen la tarea de hacer el llamado de todo lo necesario para ejecutar todos estos elementos que hemos relatado hasta ahora.

main.py:

Esta clase actúa como el punto de entrada principal para hacer las simulaciones pertinentes. Se diseñó para pruebas masivas, automatiza la lectura de parámetros desde archivos de hojas de cálculo, gestiona la carga en memoria de las matrices de probabilidad de transición (TPM) y controla el tiempo de ejecución mediante aislamiento de procesos.

```
def convertir_a_binario(texto: str, n_bits: int) -> str
```

Funciona como nuestro parser de un formato a otro, si en el excel que estamos utilizando se utilizan las letras en lugar de los números, esta función rápidamente hace el traspaso de esas letras a números en binario, representando el sistema

```
def resolver_tpm_path(estado_inicio: str, pagina: str = "A") ->
Path
```

Este tiene la función de buscar el archivo de TPM necesario para ejecutar la red necesaria, si por ejemplo tenemos sistemas de 10 nodos que encuentre el archivo específico para N10A.csv y así poder evaluar correctamente.

```
def asegurar_tpm(n_nodos: int, pagina: str) -> Path
```

En caso de no encontrar la TPM para el sistema a evaluar se hace el llamado al manager para que haga el proceso de crear dicha red para poder hacer las pruebas correspondientes.

```
def _estrategia_desde_env()
```

Se inicializa la estrategia solicitada, normalmente está configurado para que funcione directamente para ejecutar estrategia geométrica, pero pueden ponerse otras estrategias, en caso de necesitarlo.

```
def _fmt_num(val)
```

Se utiliza para formatear el número que nos da el programa para que al copiarlo en el excel como resultado sea un valor numérico y no uno de tipo de texto.

```
def _resolver_prueba(
    estado, condiciones, alcance, mecanismo, tpm, estrategia_cls
) -> dict
```

```
def _worker_prueba(args, queue)
```

Estos dos instancian de forma concurrente el cómo analizar el excel para realizar las pruebas que se piden, se encargan de ir directamente a la casilla donde se encuentra un ejercicio y obtener la respuesta a ese ejercicio específico, se tiene un timeout para evitar que se cuelgue la computadora o el propio programa.

```
def _fila_resumen(resultado: dict) -> dict
```

Esta función se encarga de recolectar todo, ordenarlo y empaquetarlo en una estructura plana, es decir, una fila horizontal de excel para los resultados que ha obtenido.

```
def ejecutar_datos_pruebas2026(
    ruta_entrada: Path,
    ruta_salida: Path | None = None,
    cantidad: int | None = None,
    hoja_filtro: str | None = None,
)
```

Coordina la ejecución del programa mediante el excel que se utiliza para las pruebas, primero se crea una copia de seguridad para no afectar el archivo original, cuando carga la matriz con Numpy, si se ve que se esta colgando pasa a usar pandas para utilizar el tipo de dato Flotante que es de 32 bits reduciendo el consumo de memoria a la mitad.

Cada vez que va solucionando el problema muestra las métricas globales con cada k y quien gana.

```
def iniciar()
```

Lee todas las funciones y se asegura de hacer una ejecución de forma segura y pueda trabajar el programa sin problema.

Para finalizar tenemos el `exec.py` que es el ejecutor del proyecto y es el que debemos ejecutar en consola para hacer el debido proceso.

```
from src.models.base.application import aplicacion
from src.main import iniciar
```

```
def main():
    """Inicializar el aplicativo."""

    aplicacion.profiler_habilitado = True
    aplicacion.pagina_sample_network = "B"

    iniciar()

if __name__ == "__main__":
    main()
```

Hace el llamado al main para la ejecución de la aplicación.

Ahora explicado GeoMIP pasaré a explicar la otra estrategia utilizada que es QNodes pero al usar una estructura similar a GeoMIP la idea es solo centrarme en los componentes donde realmente se ve unos cambios que modifican el comportamiento del mismo ya que esta estrategia se utiliza principalmente para realizar una comparación al grueso principal GeoMIP y su funcionamiento.

Para comenzar voy a empezar explicando cómo se hace el manejo y el llamado a la estrategia en la carpeta de `controllers`.

`Manager.py`:

Bueno cuenta con la misma funcionalidad de GeoMIP que es el encargado de manejar toda esa capa de saber cómo se interactúa con las carpetas, donde encontrar las TPM' s como generarlas en caso de tener que usarlas y hacer ese cálculo de los límites en la memoria.

```
def cargar_red(self) -> np.ndarray
```

Carga los arreglos bidimensionales de la TPM desde el disco duro utilizando una función optimizada, esta consume el archivo CSV mapeado dinámicamente por la propiedad de la otra función del nombre de la TPM y utiliza un delimitador de dos puntos, para reconstruir el tensor de estados en la memoria RAM corporativa.

```
def generar_red(
    self,
    dimensiones: int,
    datos_discretos: bool = True,
    pagina: str | None = None,
    auto_confirm: bool = False,
) -> str | None
```

Sigue actuando como la interfaz del GeoMIP, es decir, cuando encuentra la TPM que se va a utilizar pues la utiliza, sino es el caso empieza a generar la red en base a la letra como se había definido en el GeoMIP.

```
def _escribir_tpm(
    self,
    dimensiones: int,
    suffix: str,
    variant_number: int,
    datos_discretos: bool,
    auto_confirm: bool,
) -> str | None
```

Esta también hace la misma funcionalidad a la hora crear dicha TPM, utiliza un motor de generación sintética, para simular redes reales, y que exista esa dependencia entre cada nodo, así se sabra quien es mas dependiente del otro, dándonos una mejor creación de las matrices, se hace la optimización de los Chunks para evitar que se colapse a la hora de generar redes muy grandes, al igual que una advertencia con el tamaño de las redes.

Pasamos ahora a las funciones del sistema en la carpeta de `funcs` donde si encontramos nuevos elementos que corresponden con la nueva estrategia de Qnodes, como veremos a continuación.

force.py:

Funciona como el motor de fuerza bruta optimizado de la estrategia QNodes. A diferencia de KGeoMIP que usa un enfoque geométrico e inteligente para saltarse combinaciones, este force está diseñado con matemáticas discretas de bajo nivel y trucos de hardware para generar combinaciones masivas a la velocidad de la luz.

```
def generar_candidatos(n_vars: int)
```

Los candidatos que se generan se hacen de manera secuencial los conjuntos de variables a condicionar, comenzando desde subestructuras pequeñas de un solo nodo hasta el sistema completo de variables.

```
def generar_subsistemas(vars: tuple[int])
```

Construye las subredes candidatas aplicando un producto cartesiano entre las combinaciones de estados temporales pasados y futuros. Este producto está acotado para evitar estados vacíos que podrían romper después la marginalización.

```
def generar_particiones(
    m: int,
    n: int,
    *,
    as_matrix: bool = False,
    as_generator: bool = True,
) -> Union[Generator[Tuple[np.ndarray, np.ndarray], None, None],
np.ndarray]
```

Esta es la función que genera los cálculos matemáticos necesarios para obtener las particiones, hace un cálculo de las combinaciones binarias para particionar el sistema, esto con tres optimizaciones de bajo nivel. Se hace primero un desplazamiento de los bits, hace un ahorro de memoria para ahorrar los ciclos de la CPU.

```
def biparticiones(
    alcances: np.ndarray,
    mecanismos: np.ndarray,
    total=None,
)
```

Genera los cortes estructurales finales cruzando los subconjuntos del alcance y el mecanismo. Utiliza un cortador para descartar la primera combinación que es la partición vacía y la última, ya que carecen de valor analítico en IIT.

```
def subconjuntos(arr: np.ndarray):
    return chain.from_iterable(combinations(arr, r) for r in
range(len(arr) + 1))
```

Esta subrutina tiene como objetivo generar de forma exacta todos los subconjuntos posibles a partir de un arreglo unidimensional de variables o nodos del sistema.

Pasamos ahora a la siguiente función que sería nuestro archivo de formato.

format.py:

Como en el sistema de GeoMIP en este caso es el que nos permite tener una mejor visualización del sistema en sí, es fundamental porque traslada las complejas particiones matemáticas abstractas e índices numéricos internos a una representación gráfica legible.


```
def fmt_biparticion_fuerza_bruta(
    parte_uno: list[tuple[int, ...], tuple[int, ...]],
    parte_dos: list[tuple[int, ...], tuple[int, ...]],
) -> str
```

Esta formatea las particiones clásicas para ponerlas en formato legible mediante el uso del abecedario, donde las letras mayúsculas representan el alcance y las letras minúsculas el mecanismo

```
def fmt_biparticion_q(
    prim: list[tuple[int, int]],
    dual: list[tuple[int, int]],
    to_sort: bool = True,
) -> str
```

Coordina la renderización simultánea de los dos bloques complementarios de una bipartición, y sirve como puente entre la función de antes y la que se viene a continuación.

```
def fmt_parte_q(
    parte: list[tuple[int, int]], a_ordenar: bool = True
) -> tuple[str, str]
```

Subrutina atómica que procesa un único bloque de variables. Discrimina los nodos en función de su variable temporal, es parecido al de arriba, si la variable está en el alcance la pondrá como mayúscula sino es el caso como minúscula.

```
def partes_a_clave(
    partes_purview: list,
    partes_mecanismo: list,
) -> list[list[tuple[int, int]]]
```

Transforma las particiones que todavía están sin analizar, en llaves de emparejamiento unificadas para el generador. Ejecuta un bucle iterativo que distribuye simultáneamente los elementos entre ambos tiempos.

```
def fmt_k_particion(partes: list[list[tuple[int, int]]]) -> str
```

Con la diferencia de los formateadores normales, este procesa arreglos dinámicos de cualquier cantidad de bloques de las k particiones, este concatena horizontalmente los encabezados superiores y las bases inferiores de múltiples bloques independientes en un solo string fluido mediante uniones de flujo de bajo impacto en RAM.

```
def fmt_geomip_k_particion(partes: list[list[tuple[int, int]]]) ->
str
```

Soluciona de manera explícita el problema de persistencia regional en Windows. Remueve los caracteres unicode complejos que corrompen la renderización en herramientas de ofimática externas como Excel para poder utilizarlo y poder leerlo mediante el código ASCII.

Pasamos ahora con la otra función que quería destacar que es el IIT, que es otro componente que usamos diferente al anterior GeoMIP.

`iit.py:`

Este componente provee las herramientas de cálculo de bajo nivel y la infraestructura algebraica compartida por todo el software. Soluciona la métrica EMD, abstrae la arquitectura del procesador y hace un mapeo de las palabras utilizadas en el proyecto.

```
def get_labels(n: int) -> tuple[str, ...]
```

Implementa una subrutina recursiva interna con otra función que nos sirve para convertir índices enteros en el formato de coordenadas alfabéticas infinitas de Excel.

```
def literales(remaining_vars: NDArray[np.int8], lowercase: bool = False)
```

Se encarga de traducir los índices numéricos de los nodos remanentes a su representación simbólica alfabética condensada es decir las letras como ABC o abc.

```
def seleccionar_emd() -> Callable[
    [NDArray[np.float32], NDArray[np.float32]],
    float,
]
```

Su rol es leer la configuración global de la aplicación y devolver la función específica de la métrica EMD que se debe aplicar en los cálculos del sistema.

```
def emd_efecto(u: NDArray[np.float32], v: NDArray[np.float32]) -> float
```

Es una de las máximas optimizaciones matemáticas del proyecto. Basándose en la premisa teórica de que para variables condicionalmente independientes la EMD se reduce a la suma de las diferencias de sus distribuciones marginales, la función busca esquivar por completo el costoso problema de optimización de transporte de la programación lineal.

```
def emd_causal(u: NDArray[np.float64], v: NDArray[np.float64]) -> float
```

Es el que busca resolver el análisis del presente hacia el pasado. Utiliza de forma dinámica la librería de Pyemd y preasigna una matriz de costos vacía utilizando la distancia de bits de Hamming para poblar la matriz reflejada simétricamente, resolviendo el transporte de probabilidad tradicional.

```
def seleccionar_distancia() -> Callable[
    [NDArray[np.float32], NDArray[np.float32]],
    float,
]
```

La función que aplica es la de escoger la métrica de distancia espacial base que el resolovedor causal necesita para construir la matriz de costos de transporte de probabilidad.

```
def count_bits(n: int) -> int
```

Cuenta de forma exacta cuántos bits están encendidos, osea que están en estado 1 dentro de la representación binaria de un número entero.

```
def reindexar(n: int) -> np.ndarray
```

El objetivo de esta función es la de decidir, con base en la configuración del entorno, qué ordenamiento de indexación numérica se debe aplicar a los arreglos de datos del sistema

```
def seleccionar_estado(subestado: np.ndarray) -> np.ndarray
```

Busca reordenar físicamente el vector de un estado binario actual, para alinearlo exactamente con la convención de ordenamiento configurada.

```
def big_endian(n: int) -> np.ndarray
```

Esta función es la encargada de generar la secuencia base de indexación lineal de los nodos bajo la convención estándar de ordenamiento superior.

```
def lil_endian(n: int) -> np.ndarray
```

Esto permite transformar arreglos de indexación lineal en mapas de memoria de orden invertido optimizados para hardware. ya que es el motor para el ordenamiento de la memoria.

```
def get_restricted_combinations(binary_str: str) -> tuple[list[str], list[str]]
```

Esta función actúa como un generador de combinaciones binarias restringidas por máscaras de bits. Su función principal es delimitar el espacio de búsqueda combinatoria para los subconjuntos del hipercubo.

```
def generate_combinations(A: str) -> list[tuple[str, str, str]]
```

Su función es dar formato de visualización a las cadenas de bits y cruzar el sistema de control A con todos los subestados válidos y restringidos de los bloques B y C.

Por último vamos a ir con el archivo más importante, que es la estrategia en sí, que sería nuestro archivo de Qnodes.

q_nodes.py:

Aquí se encuentra la estrategia diferente a GeoMIP, aquí entenderemos como aplica las particiones de $k = 2$ hasta $k = 5$ mediante una forma muy parecida a la fuerza bruta pero sin llegar a ese punto, y de manera más optimizada dándonos los resultados adecuados.

```
def __init__(self, tpm: np.ndarray)
```

Inicializa el solucionador utilizando el constructor de la clase abstracta superior SIA. Levanta automáticamente una sesión de perfilado de rendimiento en hardware para auditorías y declara múltiples diccionarios de caché críticos, como la memoria delta y la memoria unión, que son memorias repetitivas para evitar los cálculos de los mismos grupos de nodos.

```
def aplicar_estrategia(
    self,
    estado_inicial: str,
    condicion: str,
    alcance: str,
    mecanismo: str,
)
```

Funciona como nuestro director de la orquesta, tiene una secuencia de pasos que hacen las siguientes acciones, primero hace un mapeo, donde se descompone el subsistema separando los nodos en variables de efecto y variables actuales, después hace una limpieza del caché, para garantizar el avalúo correcto de los datos. Empieza a llamar el motor algorítmico para obtener la bipartición base, y después empieza a calcular la información de lo que sería partirlo en mayores k .

```
def _resultado_biparticion(self, mip)
```

Recupera de forma directa el resultado óptimo de un corte binario $k = 2$ usando como índice la clave mip sobre el diccionario de memorización de los grupos de candidatos. Extrae la pérdida y las distribuciones.

```
def _mejor_k_particion(self, k: int)
```

Actúa como el optimizador local para un nivel k específico. empieza a recopilar los candidatos registrados en el árbol estructural y después los expande, después de expandirlos, hace un congelamiento en las tuplas para evitar volver a realizar los mismos estados o equivalentes. si pasa ese filtro hace un llamado a una función para realizar el corte.

```
def _refinar_cortes_hacia_k(self, k: int) -> list[list]
```

Hace una rutina heurística fundamental. Ya que construye una lista de estructuras de agrupamiento llamadas bases tomando la bipartición reina y los cortes históricos de niveles inferiores e inicia un bucle de división progresiva.

```
def _dividir_grupo_greedy(
    self, nodos: list[tuple[int, int]]
```

```
) -> tuple[list[tuple[int, int]], list[tuple[int, int]]]
```

Ejecuta un corte binario inteligente sobre una lista de variables utilizando un enfoque constructivo, primero ordenada de manera determinista los nodos, hace un bucle que aplica la técnica voraz, para obtener los nodos de menor costo de interacción.

```
def _evaluar_grupos_k(self, grupos)
```

Cuantifica el impacto de una k-partición específica en el sistema. Descompone los grupos analíticos, se llama el resolutor estructural del subsistema para inyectar los cortes en los tensores de probabilidad.

```
def _grupos_a_partes(self, grupos)
```

Transforma listas de tuplas abstractas en arreglos físicos empaquetados de NumPy con tipado optimizado para el menor consumo.

```
def algorithm(self, vertices: list[tuple[int, int]])
```

Implementa el ciclo principal. Permite registrar el corte global inicial y entra en un bucle de contracción de $n - 1$ pasos. Después hace una aglomeración y atajos de parada ya que inicia las listas de ciclo. De ahí empieza a evaluar los costos marginales de absorción. Si en algún punto se detecta que la EMD del conjunto aislado es idéntica a cero, el algoritmo activa un criterio de parada temprana, asume que se ha encontrado un subconjunto totalmente independiente y retorna de inmediato la clave, ahorrando ciclos de CPU. Por último, Al final de cada ciclo, toma el último elemento y los fusiona en una estructura empaquetada y actualiza la lista de vértices activos, reduciendo la dimensionalidad aparente del sistema en una unidad y registrando el estado en el árbol jerárquico.

```
def funcion_submodular(
    self, deltas: Union[tuple, list[tuple]], omegas:
list[Union[tuple, list[tuple]]]
)
```

Es la función con mayor carga matemática ya que permite resolver el costo energético de las estructuras de nodos basándose en dos niveles de caché, El caché delta y el cache unión, la delta busca aislar los parámetros, para definir sus índices y mirar si mediante el EMD ya se ha calculado su corte. si no es el caso hace el llamado a la función de bipartir, y lo indexa en una tupla. El caché de unión busca mirar como es la combinatoria de los elementos. hace un llamado a los retornos del transporte de las EMD's.

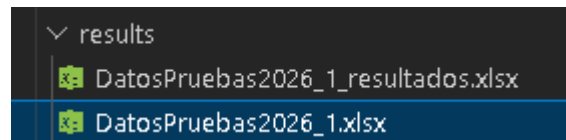
```
def definir_clave(
    self,
    conjunto: Union[tuple[int, int], list[tuple[int, int]]],
)
```

Actúa como un normalizador topológico. Recibe un nodo o colección de nodos, extrae sus marcas y los segrega en los arreglos del Mecanismo y del alcance. Cuando finaliza, aplica un ordenamiento explícito sobre ambas dimensiones.

Con esto finalmente se ha terminado de definir tanto Qnodes con el KGeoMIP, esto corresponde a un análisis completo de cada función y acción que deben realizar los algoritmos para que hagan lo que deben hacer.

8. Resultados Experimentales

Para la evaluación de nuestro algoritmo KGeoMip se utilizó un dataset que compartió nuestra profesora meses atrás, donde se hacían varias pruebas con distintas determinaciones, este banco de pruebas estandarizado se denominó [DatosPruebas2026_1.xlsx](#) este se encuentra ubicado en el proyecto en la carpeta de [results](#), donde ahí mismo se genera el excel como una copia de este, pero con los resultados escritos en el.



Cuando entramos a este excel encontramos como primer elemento en la primera hoja que nos dice la plataforma o computadora que estamos utilizando para estas pruebas, para estas pruebas se utilizaron dos Computadores fundamentalmente uno que cuenta con las siguientes especificaciones:

Primer Computador:

Amd Ryzen 5 5500U 2,10 GHZ

16 Gb de RAM

Amd Radeon Graphics (Tarjeta de Video Integrada)

Segundo Computador:

Intel Core I7-13620H 2,40 GHZ

16 Gb de RAM

Intel UHD Graphics

En estos dos computadores se realizaron las pruebas exhaustivamente para cada hoja del excel, estos computadores cuentan con un sistema operativo de Windows, pero ahora, ¿Que contiene este excel?

Este dataset está compuesto por múltiples hojas de cálculo que representan sistemas lógicos donde se pueden observar varios detalles fundamentales que veremos a continuación.

Estado inicial	10000000000000000000000000000000
Sistema:	ABCDEFGHIJKLMN OPQRSTUVWXYZ
Sistema Candidato:	ABCDEFGHIJKLMN OPQRSTUVWXYZ

Este dataset nos brinda primeramente un estado inicial, el sistema que va a utilizar, el candidato.


```

Prueba #10 | alcance=1111111110 mecanismo=0111111111
CRITICAL (10:47:47): Original creado.
CRITICAL (10:47:47): Original:
CRITICAL (10:47:47): Candidato creado.
CRITICAL (10:47:47): Subsistema creado.
CRITICAL (10:47:47): empieza.
K=2
| A,H || B,C,D,E,F,G,I |
| b,c || d,e,f,g,h,i,j |
0.4409356
K=3
| A,H || B,D,I || C,E,F,G |
| b,c || d,e,f || g,h,i,j |
0.6938584
K=4
| H || C,E,F,G || A,B || D,I |
| b || g,h,i,j || c,d || e,f |
0.69665444
K=5
| A,H || B,I || E,F,G || D || C |
| b,c || d,e || g,h || f || i,j |
0.7080525
-> Mejor k=2 perdida=0.4409 tiempo=44.523s

```

En la imagen, vemos un ejemplo con GeoMIP, donde en la consola nos muestra cómo corre las operaciones, y nos mencionan varios mensajes, primero la prueba que está realizando de la hoja de excel, y la mejor partición que saco según el k utilizado, al final nos dice la mejor partición entre las 5 que hubieron, y fue con 2 en este caso, cuando acabe las 50 pruebas ahí es donde se almacenan los valores.

Ahora en la consola de Qnodes tenemos lo siguiente:

Q-Nodes – Evaluación k-particiones (k = 2 ... 5)

K=2 ◀ MEJOR

| A,B || C |

| c || a,b |

$\phi = 0.5000000$ tiempo = 0.0100s

K=3

| $\emptyset$ || A,B || C |

| a || b,c || $\emptyset$ |

$\phi = 0.5000000$ tiempo = 0.0190s

K=4

| $\emptyset$ || A || B || C |

| a || b,c || $\emptyset$ || $\emptyset$ |

$\phi = 0.7500000$ tiempo = 0.0100s

K=5

| $\emptyset$ || A || $\emptyset$ || B || C |

| a || c || b || $\emptyset$ || $\emptyset$ |

$\phi = 1.2500000$ tiempo = 0.0030s

Vemos cómo también hace su trabajo sacando las particiones según su k, nos muestra cómo fueron los cortes, la pérdida que esta estrategia genera, y el tiempo utilizado, para estas prácticas, se usó para el KGeoMIP el primer computador y el segundo computador para hacer los cálculos de Qnodes.

Después vemos como se refleja en nuestro excel, cuando acaba, como deja las particiones hechas y realizadas.

Geometric		
Partición	Pérdida	Tiempo
I A,B,C,D,E,F,G,H,I a b,c,d,e,f,g,h,i,j	0,74	1,55
A,B,C,D,G,H,I,J E,F a,b,c,d,e,f,g,h i	0,69	0,41
H A,B,C,D,E,F,G,I,J b c,d,e,f,g,h,i,j	0,68	0,40
C,D,F,G,H,I A,B,E,J d,e,f,i b,c,g,h	0,39	0,11
F A,B,C,D,E,G,H,I,J b,d,e,i a,g,h	0,24	0,04
A,G,H,I B,C,D,E,F,J a,c,e,i g	0,23	0,02
B,F,G,I,J A,C,D,E,H b,f,h,i d	0,19	0,02
I A,B,C,D,E,F,G,H a b,c,d,e,f,g,h,i,j	0,68	1,32
B A,C,D,E,F,G,H,I a b,c,d,e,f,g,h,i	0,49	0,35
A,H B,C,D,E,F,G,I b,c d,e,f,g,h,i,j	0,44	0,38
B,F,H,I A,C,D,E,G c,d,f,h b,e,g,i	0,28	0,10
C,F,G,I A,B,D,E,H a,b,d,e g,h,i	0,31	0,03
A,G,H,I B,C,D,E,F a,c,e,i g	0,14	0,01
B,F,G,I A,C,D,E,H b,f,h,i d	0,18	0,02
I B,C,D,E,F,G,H,I a b,c,d,e,f,g,h,i,j	0,74	1,33
B,C,D,E,G,H,I,J F a,b,c,d,e,f,g,h i	0,53	0,36
H B,C,D,E,F,G,I,J b c,d,e,f,g,h,i,j	0,57	0,37
B,C,D,F,G,H,I,J E b,c,d,e,f,g,h i	0,39	0,11
F B,C,D,E,G,H,I,J b,d,e,i a,g,h	0,20	0,03
G,H,I B,C,D,E,F,J a,c,e,i g	0,22	0,01
B,F,G,I,J C,D,E,H b,f,h,i d	0,13	0,02

Aquí vemos que en el archivo copia que creo se almacena cada vez que termina las pruebas de una cantidad de nodos determinada, nos almacena tanto la pérdida como el tiempo que se tardó para encontrarla. Miremos los resultados obtenidos pero esta vez por el Qnodes.

QNodes		
Partición	Pérdida	Tiempo
A,B,C,D,E,G,H,I,J F a,b,c,d,e,f,g,h,i,j ∅	0,0341	0,4954
A,B,D,E,F,G,H,I,J C a,b,c,d,e,f,g,h,i ∅	0,0047	0,3570
A,C,D,E,F,G,H,I,J B b,c,d,e,f,g,h,i,j ∅	0,0206	0,3603
A,B,C,E,F,G,H,I,J D b,c,d,e,f,g,h,i ∅	0,0204	0,2559
A,C,D,E,F,G,H,I,J B a,b,d,e,g,h,j ∅	0,0244	0,2304
A,B,C,E,F,G,H,I,J D a,c,e,g,i ∅	0,0002	0,1437
A,B,D,E,F,G,H,I,J C b,d,f,h,j ∅	0,0056	0,1472
A,B,C,D,E,G,H,I F a,b,c,d,e,f,g,h,i,j ∅	0,0341	0,2990
A,B,D,E,F,G,H,I C a,b,c,d,e,f,g,h,i ∅	0,0047	0,2261
A,C,D,E,F,G,H,I B b,c,d,e,f,g,h,i,j ∅	0,0206	0,3305
A,B,C,E,F,G,H,I D b,c,d,e,f,g,h,i ∅	0,0204	0,4048
A,C,D,E,F,G,H,I B a,b,d,e,g,h,j ∅	0,0244	0,2466
A,B,C,E,F,G,H,I D a,c,e,g,i ∅	0,0002	0,1515
A,B,D,E,F,G,H,I C b,d,f,h,j ∅	0,0056	0,1424
B,C,D,E,G,H,I,J F a,b,c,d,e,f,g,h,i,j ∅	0,0341	0,3836
B,D,E,F,G,H,I,J C a,b,c,d,e,f,g,h,i ∅	0,0047	0,3719
B C,D,E,F,G,H,I,J ∅ b,c,d,e,f,g,h,i,j	0,0206	0,4263
B,C,E,F,G,H,I,J D b,c,d,e,f,g,h,i ∅	0,0204	0,2778

Por último vemos al final como nos deja la mejor partición encontrada por el programa.

MEJOR $k=2$	0,48263147473
MEJOR $k=5$	0,61217606068
MEJOR $k=2$	0,47704291344
MEJOR $k=2$	0,34411507845
MEJOR $k=2$	0,25218433142
MEJOR $k=2$	0,15027740598
MEJOR $k=2$	0,15567329526
MEJOR $k=2$	0,34252178665

Esto nos muestra que regularmente partir en 2 únicamente suele ser lo mejor, pero hay ocasiones donde se partió por 5 y fue mejor que hacer la bipartición, por lo que aunque parezca que con una bipartición se haría la menor pérdida, en muchas ocasiones realmente eso no es lo que ocurre.

9. Limitaciones y Trabajo Futuro

A pesar de obtener resultados favorables en el enfoque geométrico para mitigar la explosión combinatoria del número de Bell, el algoritmo presenta algunos problemas vinculados a la arquitectura y a la naturaleza que tiene este problema matemático.

Es que principalmente el problema es de su dimensionalidad 2^n , aunque el espacio de búsqueda de particiones se transformó en un problema geométrico indexado de tamaño polinomial, la representación del sistema, es decir los TPM's, se almacena en tensores de estados de tamaño exponencial 2^n . Para sistemas que superen las $n = 20$ variables, el tamaño físico de los arreglos desborda la memoria caché e indexación de CPUs convencionales, limitando la escalabilidad del software en entornos locales.

Otra limitación que encontramos fue el generador de candidatos ya que opera bajo la premisa de que existen asimetrías o relieves informativos en el hipercubo de estados. En topologías de red con un acoplamiento total, simétrico y uniforme, el relieve geométrico se aplanan. En este escenario específico, las heurísticas pierden su ventaja selectiva, comportándose temporalmente de forma cercana a una exploración que intenta no ser exhaustiva pero por otro lado corre con ese problema que termina llegando casi a ese nivel de revisar todos los casos.

También se genera una dependencia de la precisión métrica, cuando se realiza el cálculo del EMD que es el componente computacionalmente más costoso. Las aproximaciones aplicadas para acelerarlo dependen de que las distribuciones marginales estén limpias; variaciones de ruido numérico extremo en los datasets de entrada pueden desviar la selección de la frontera de corte óptima.

Durante el diseño e implementación de KGeoMIP, se implementaron algunos simplificadores que delimitan el alcance del software y podrían restringir su aplicación en contextos industriales o dinámicas de sistemas diferentes, un caso serían los sistemas estacionarios y discretos, porque si asumimos que las redes lógicas operan bajo estados discretos es decir binarios y dinámicas estacionales descritas por matrices de probabilidad de transición estáticas en el tiempo. El algoritmo no está diseñado para procesar flujos de datos continuos en tiempo real o series temporales dinámicas sin un paso de discretización previo.

El sistema asume que el funcionamiento concreto en el momento es usar el lenguaje en español y como los caracteres operan para este idioma. Si en algún momento se despliega en entornos corporativos con configuraciones anglosajonas puras, el formateador numérico mutará los valores flotantes a cadenas de texto no graficables, generando errores en la ejecución y funcionamiento normal de la aplicación.

La arquitectura de paralelización que se adoptó asume que el hardware anfitrión dispone de múltiples núcleos físicos independientes y memoria RAM simétrica para contener el proceso de buena manera. Si no contamos con el hardware adecuado, el programa puede colgarse con facilidad ya que se hizo específicamente para computadores que tienen unos componentes mejores a los que suelen promediar en el mercado.

Algunas mejoras Potenciales que se podrían aplicar a corto plazo puede ser elevar la robustez y optimizar el rendimiento del software en el ciclo de desarrollo inmediato, se identifican las siguientes líneas de mejora técnica

Transición a Tipado Estricto Estático con algún otro lenguaje para intentar evitar los cuellos de botella matemáticos como por ejemplo el bucle de acumulación del hipercubo en el cálculo del costo y otras funciones vectorizadas, esto sería un avance importante porque Esto eliminaría por completo la sobrecarga del intérprete de Python, permitiendo procesar redes más grandes en una fracción del tiempo actual.

Otra mejora que se podría implementar es el reemplazo del cálculo nativo de la EMD por resolvers de optimización matemática de grado industrial, ya que estos motores aprovechan instrucciones vectoriales de hardware avanzado y optimizan la asignación de transporte de probabilidad con mayor velocidad.

Se puede intentar una refactorización de caché distribuida, en pocas palabras, sería sustituir la estructura interna de la memoria de las particiones, porque en este momento funciona como un diccionario local en RAM por un motor de almacenamiento en caché key-value en memoria externa. Ya que nos permitiría persistir las firmas de particiones ya evaluadas entre diferentes campañas de pruebas de forma permanente, eliminando los tiempos de calentamiento del script en ejecuciones sucesivas.

Este proyecto nos abre preguntas científicas y líneas de investigación de alto valor para la comunidad de la Teoría de la Información Integrada, ya que es un campo con mucha complejidad para utilizar y mejorar con el pasar del tiempo, además haciendo uso de las herramientas enseñadas en clase como la heurística seguramente en años venideros podremos obtener mejores resultados en este tipo de problemas y tener alcances muchos mayores, ya que la invención de la IA y como sigue mejorando cada vez puede que nos brinde una solución mucho mejor en el paso del tiempo y se pueden hacer implementaciones donde se evalúen muchos más nodos y con mas k-particiones en tiempos menores o aceptables para nosotros.

10. Apéndices Técnicos

Para la realización de este proyecto se hizo un uso pertinente del Material de Clase y Guías por parte de la docente.

Dentro del proyecto se hizo uso de varios temas utilizados para la búsqueda de la optimización y realización del algoritmo funcional de nuestro proyecto.

Primero por la parte del Divide Y Vencerás usamos como base el contenido compartido por ella en la plataforma de Moodle.

[Sorting \(Bubble, Selection, Insertion, Merge, Quick, Counting, Radix\) - VisuAlgo](#)

También se hizo uso de la sección de los algoritmos voraces mediante el entendimiento de los talleres que se proporcionaron.

[59G8F-2: Taller Voraz](#)

[59G8F-2: Taller Voraz #2](#)

También se hizo mucho uso de los algoritmos Heurísticos y Aproximados, aprovechando la Gema compartida por la docente para el entendimiento y funcionamiento de las heurísticas y como aplicarlas en el proyecto.

[Microsoft PowerPoint - ALGOII-clase2](#)

[Tema9Heurísticas.ppt](#)

[59G8F-2: Gema "Algoritmos heurísticos y aproximados"](#)

También se hizo uso de los recursos compartidos para la realización del proyecto, ya que fue la base fundamental para la realización del problema que serían los PDF's compartidos del funcionamiento de GeoMIP y el documento de que lo que tiene que hacer el proyecto.

Se investigó mucho mediante videos en internet para la aplicación de heurísticas a los códigos, esto mediante videos para saber cómo se pueden integrar en la materia.

[Heuristic Algorithm for Simple Code Generation](#)

[The 10 Usability Heuristics - YouTube](#)

Por último se hizo uso del apartado bibliográfico manifestado por la profesora con uno de sus libros sobre la introducción a los algoritmos y uno que encontramos que nos ayudó todavía más a entender las heurísticas.

Tessore, Carlos & Petrella, Carlos. (2024). LAS HEURÍSTICAS.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Por último una tabla más general con los resultados obtenidos por GeoMIP y Qnodes con 10, 15, 20 y 22 nodos

PRUEBAS 4-PARTICIONES		
Geometric		
Partición	Pérdida	Tiempo
C,D,E,G,H,I,J F B A a,b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,14	0,09
A,B,D,E,G,I,J C F H a,b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,07	0,09
C,D,E,G,H,I,J B A F b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,16	0,08
A,B,E,F,G,I,J D C H b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,10	0,09
C,D,E,F,G,I,J B H A a,b,d,e,g,h,i,j ∅ ∅ ∅	0,09	0,08
A,B,C,E,H,I,J D F G a,c,e,g,i ∅ ∅ ∅	0,02	0,07
b,d,f,h,i ∅ ∅ ∅	0,03	0,08
C,D,E,G,H,I F B A a,b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,14	0,07
A,B,D,E,G,I C F H a,b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,07	0,06
C,D,E,G,H,I B A F b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,16	0,05
A,B,E,F,G,I D C H b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,10	0,06
C,D,E,F,G,I B H A a,b,d,e,g,h,i,j ∅ ∅ ∅	0,09	0,05
A,B,C,E,H,I D F G a,c,e,g,i ∅ ∅ ∅	0,02	0,04
A,D,F,G,H,I C B E b,d,f,h,i ∅ ∅ ∅	0,03	0,04
C,D,E,G,H,I F B I a,b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,20	0,06
B,D,E,G,I,J C F H a,b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,07	0,06
D,E,G,H,I,J B F C b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,20	0,06
B,E,F,G,I,J D C H b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,10	0,05
C,D,E,F,G,I B H I a,b,d,e,g,h,i,j ∅ ∅ ∅	0,09	0,04
a,c,e,g,i ∅ ∅ ∅	0,02	0,05
D,E,G,H,I,J C B E b,d,f,h,i ∅ ∅ ∅	0,03	0,04
a,b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,20	0,05
B,D,E,G,I C F H a,b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,07	0,04
D,E,G,H,I B F C b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,20	0,04
b,c,d,e,f,g,h,i,j ∅ ∅ ∅	0,10	0,04
a,b,d,e,g,h,i,j ∅ ∅ ∅	0,10	0,03
B,C,E,H,I D F G a,c,e,g,i ∅ ∅ ∅	0,02	0,03

PRUEBAS 5-PARTICIONES

Geometric			
Partición	Pérdida	Tiempo	
B,C,D,F,G,H,I,J,K,L,O N A E M a,b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,08	1,80
B,C,D,F,G,H,I,J,K,L,N,O A G M E a,b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,05	1,34
A,B,C,D,G,I,K,L,M,N,O E I H F b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,12	1,16
A,C,D,F,G,H,I,J,K,L,M,N,O I E G B b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,05	1,04
A,B,C,D,F,G,H,I,J,K,L,O N M E G a,b,d,e,g,h,i,k,m,n,o Ø Ø Ø Ø		0,08	0,65
A,B,D,F,G,H,I,J,L,N,O K M C E a,c,e,g,i,k,m,o Ø Ø Ø Ø		0,01	0,61
A,B,C,E,F,I,J,K,M,N,O H L G D b,d,f,h,i,l,n Ø Ø Ø Ø		0,02	0,55
B,C,D,F,G,H,I,J,K,L N A E M a,b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,08	1,90
B,C,D,F,G,H,I,J,K,L,N A G M E a,b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,05	1,34
b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,12	1,14
A,C,D,F,G,H,I,J,K,L,M,N I E G B b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,05	1,02
A,B,C,D,F,G,H,I,J,K,L N M E G a,b,d,e,g,h,i,k,m,n,o Ø Ø Ø Ø		0,08	0,71
a,c,e,g,i,k,m,o Ø Ø Ø Ø		0,01	0,60
A,B,C,E,F,I,J,K,M,N H L G D b,d,f,h,i,l,n Ø Ø Ø Ø		0,02	0,56
B,C,F,G,H,I,J,K,L,O N E M D a,b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,12	1,92
B,C,D,F,G,H,I,J,K,N,O G M E L a,b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,08	1,27
b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,12	1,12
b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,05	1,04
B,C,D,F,G,H,I,J,K,L,O N M E G a,b,d,e,g,h,i,k,m,n,o Ø Ø Ø Ø		0,08	0,71
B,D,F,G,H,I,J,L,N,O K M C E a,c,e,g,i,k,m,o Ø Ø Ø Ø		0,01	0,55
B,C,E,F,I,J,K,M,N,O H L G D b,d,f,h,i,l,n Ø Ø Ø Ø		0,02	0,56
B,C,F,G,H,I,J,K,L N E M D a,b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,12	1,67
B,C,D,F,G,H,I,J,K,N G M E L a,b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,08	1,00
B,C,D,G,I,K,L,M,N E I H F b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,12	0,94
b,c,d,e,f,g,h,i,j,k,l,m,n,o Ø Ø Ø Ø		0,05	0,86
B,C,D,F,G,H,I,J,K,L N M E G a,b,d,e,g,h,i,k,m,n,o Ø Ø Ø Ø		0,08	0,55
B,D,F,G,H,I,J,L,N K M C E a,c,e,g,i,k,m,o Ø Ø Ø Ø		0,01	0,46
B,C,E,F,I,J,K,M,N H L G D b,d,f,h,i,l,n Ø Ø Ø Ø		0,02	0,45

PRUEBAS 3-PARTICIONES

Geometric			
Partición	Pérdida	Tiempo	
a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t Ø Ø		0,04	32,51
A,B,C,D,F,G,H,I,J,K,L,M,N,O,Q,R,S,T P E a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s Ø Ø		0,04	10,45
A,B,C,E,F,G,H,I,J,K,L,M,N,O,P,Q,S,T B D b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t Ø Ø		0,02	9,55
A,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,S,T B P b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s Ø Ø		0,00	5,34
A,B,C,D,E,F,G,H,I,J,K,M,N,O,P,Q,R,S,T I L a,b,d,e,g,h,i,k,m,n,p,q,s,t Ø Ø		0,03	0,71
A,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S B T a,c,e,g,i,k,m,o,g,s Ø Ø		0,00	0,32
B,C,D,E,F,G,H,I,J,K,L,M,N,P,Q,R,S,T O A b,d,f,h,i,l,n,p,r,t Ø Ø		0,00	0,33
A,B,C,E,F,G,H,I,J,K,L,M,N,O,Q,R,S D P a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t Ø Ø		0,04	26,90
A,B,C,D,F,G,H,I,J,K,L,M,N,O,Q,R,S P E a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s Ø Ø		0,04	9,80
A,B,C,E,F,G,H,I,J,K,L,M,N,O,P,Q,S B D b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t Ø Ø		0,02	9,07
A,C,D,E,F,G,H,I,J,K,L,M,N,O,Q,R,S B P b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s Ø Ø		0,00	4,86
A,B,C,D,E,F,G,H,I,J,K,M,N,O,P,Q,R,S I L a,b,d,e,g,h,i,k,m,n,p,q,s,t Ø Ø		0,03	0,61
A,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,S B R a,c,e,g,i,k,m,o,g,s Ø Ø		0,00	0,25
B,C,D,E,F,G,H,I,J,K,L,M,N,P,Q,R,S O A b,d,f,h,i,l,n,p,r,t Ø Ø		0,00	0,28
B,C,E,F,G,H,I,J,K,L,M,N,O,Q,R,S,T D P a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t Ø Ø		0,04	29,41
B,C,D,F,G,H,I,J,K,L,M,N,O,Q,R,S,T P E a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s Ø Ø		0,04	11,17
B,C,E,F,G,H,I,J,K,L,M,N,O,P,Q,S,T B D b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t Ø Ø		0,02	9,46
C,D,E,F,G,H,I,J,K,L,M,N,O,Q,R,S,T B P b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s Ø Ø		0,00	4,47
B,C,D,E,F,G,H,I,J,K,M,N,O,P,Q,R,S,T I L a,b,d,e,g,h,i,k,m,n,p,q,s,t Ø Ø		0,03	0,66
C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S B T a,c,e,g,i,k,m,o,g,s Ø Ø		0,00	0,32
B,C,D,E,F,G,H,I,J,L,M,N,P,Q,R,S,T O K b,d,f,h,i,l,n,p,r,t Ø Ø		0,00	0,25
a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t Ø Ø		0,04	26,65
B,C,D,F,G,H,I,J,K,L,M,N,O,Q,R,S P E a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s Ø Ø		0,04	10,50
B,C,E,F,G,H,I,J,K,L,M,N,O,P,Q,S B D b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t Ø Ø		0,02	9,06
C,D,E,F,G,H,I,J,K,L,M,N,O,Q,R,S B P b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s Ø Ø		0,00	4,28
B,C,D,E,F,G,H,I,J,K,M,N,O,P,Q,R,S I L a,b,d,e,g,h,i,k,m,n,p,q,s,t Ø Ø		0,03	0,57
C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,S B R a,c,e,g,i,k,m,o,g,s Ø Ø		0,00	0,25
B,C,D,E,F,G,H,I,J,L,M,N,P,Q,R,S O K b,d,f,h,i,l,n,p,r,t Ø Ø		0,00	0,24

PRUEBAS 5-PARTICIONES			
Geometric			
Partición		Pérdida	Tiempo
A,C,D,E,F,G,H,I,J,K,L,M,O,P,Q,R,S,T,V	B U Q N a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,y 0 0 0 0		0,08 337,25
A,B,C,D,E,F,G,I,J,K,L,M,N,O,P,Q,S,T,U	H V I B a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u 0 0 0 0		0,21 79,63
A,C,E,F,G,H,I,K,L,M,N,O,P,Q,R,T,U,V	B S I D b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,y 0 0 0 0		0,10 75,55
b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u 0 0 0 0			0,08 40,41
A,C,D,E,F,G,H,I,J,K,L,M,O,P,Q,R,S,U N V B T a,b,d,e,g,h,i,j,k,m,n,p,q,g,s,t,v 0 0 0 0			0,06 3,35
A,C,E,F,G,H,I,K,L,M,N,O,P,Q,R,S,U,V I T D B a,c,e,g,i,k,m,o,g,s,u 0 0 0 0			0,00 1,12
A,B,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,T,V S C U E b,d,f,h,i,l,n,p,r,t,v 0 0 0 0			0,00 1,14
A,C,D,E,F,G,H,I,J,K,L,M,O,P,R,S,T	B U Q N a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,y 0 0 0 0		0,08 335,13
A,B,C,D,E,F,G,I,K,L,M,N,O,P,Q,S,U	H I B T a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u 0 0 0 0		0,23 75,77
A,C,E,F,G,H,I,K,L,M,N,O,P,Q,R,T,U	B S I D b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,y 0 0 0 0		0,10 75,70
A,B,C,D,E,F,H,K,L,M,O,P,Q,R,S,T,U	G N I I b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u 0 0 0 0		0,08 38,31
A,C,D,E,F,G,H,I,J,K,L,M,O,P,R,S,U N B T Q a,b,d,e,g,h,i,j,k,m,n,p,q,g,s,t,v 0 0 0 0			0,06 2,86
A,C,E,F,G,H,I,K,L,M,N,O,P,Q,R,S,U I T D B a,c,e,g,i,k,m,o,g,s,u 0 0 0 0			0,00 1,10
A,B,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,T S C U E b,d,f,h,i,l,n,p,r,t,v 0 0 0 0			0,00 1,07
a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,y 0 0 0 0			0,08 323,92
a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u 0 0 0 0			0,21 74,63
C,E,F,G,H,I,K,L,M,N,O,P,Q,R,T,U,V	B S I D b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,y 0 0 0 0		0,10 72,16
b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u 0 0 0 0			0,08 41,11
a,b,d,e,g,h,i,k,m,n,p,q,g,s,t,v 0 0 0 0			0,06 2,71
C,E,F,G,H,I,K,L,M,N,O,P,Q,R,S,U,V I T D B a,c,e,g,i,k,m,o,g,s,u 0 0 0 0			0,00 1,07
B,D,E,F,G,H,I,K,L,M,N,O,P,Q,R,T,V S C U E b,d,f,h,i,l,n,p,r,t,v 0 0 0 0			0,00 1,10
a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,y 0 0 0 0			0,08 315,70
B,C,D,E,F,G,I,K,L,M,N,O,P,Q,S,U	H I B T a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u 0 0 0 0		0,23 70,35
b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,y 0 0 0 0			0,10 76,63
B,C,D,E,F,H,K,L,M,O,P,Q,R,S,T,U	G N I I b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u 0 0 0 0		0,08 38,47
C,D,E,F,G,H,I,J,K,L,M,O,P,R,S,U N B T Q a,b,d,e,g,h,i,j,k,m,n,p,q,g,s,t,v 0 0 0 0			0,06 2,33
C,E,F,G,H,I,K,L,M,N,O,P,Q,R,S,U I T D B a,c,e,g,i,k,m,o,g,s,u 0 0 0 0			0,00 1,04
B,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,T S C U E b,d,f,h,i,l,n,p,r,t,v 0 0 0 0			0,00 1,05

Qnodes

QNodes			
Partición	Pérdida	Tiempo	
A C,D,E,G,H,I,J B F I 0 a,b,c,d,e,f,g,h,i,j 0 0	0,1378	0,3483	
A,B,D,E,G,I,J C F H I a,b,c,d,e,f,g,h,i 0 0 0	0,0656	0,2249	
A B C,D,E,G,H,I,J F I 0 0 b,c,d,e,f,g,h,i,j 0	0,1610	0,2510	
A,B,E,F,G,I,J C D H I b,c,d,e,f,g,h,i 0 0 0	0,0985	0,1754	
A C,D,E,F,G,I,J B H I 0 a,b,d,e,g,h,j 0 0	0,0874	0,1370	
A,B,C,E,H,I,J D F G I a,c,e,g,i 0 0 0	0,0161	0,0970	
A,D,F,G,H,I,J B C E I b,d,f,h,j 0 0 0	0,0283	0,0905	
A C,D,E,G,H,I B F I 0 a,b,c,d,e,f,g,h,i,j 0 0	0,1378	0,2100	
A,B,D,E,G,I C F H I a,b,c,d,e,f,g,h,i 0 0 0	0,0656	0,1380	
A B C,D,E,G,H,I F I 0 0 b,c,d,e,f,g,h,i,j 0	0,1610	0,2248	
A,B,E,F,G,I C D H I b,c,d,e,f,g,h,i 0 0 0	0,0985	0,2367	
A C,D,E,F,G,I B H I 0 a,b,d,e,g,h,j 0 0	0,0874	0,1359	
A,B,C,E,H,I D F G I a,c,e,g,i 0 0 0	0,0161	0,0859	
A,D,F,G,H,I B C E I b,d,f,h,j 0 0 0	0,0283	0,0876	
C,D,E,G,H,I B F I I a,b,c,d,e,f,g,h,i,j 0 0 0	0,1995	0,2345	
B,D,E,G,I,J C F H I a,b,c,d,e,f,g,h,i 0 0 0	0,0656	0,2065	
B D,E,G,H,I,J C F I 0 b,c,d,e,f,g,h,i,j 0 0	0,1971	0,2339	
B,E,F,G,I,J C D H I b,c,d,e,f,g,h,i 0 0 0	0,0985	0,1645	
C,D,E,F,G,I B H I I a,b,d,e,g,h,j 0 0 0	0,0932	0,1248	
B,C,E,H,I,J D F G I a,c,e,g,i 0 0 0	0,0161	0,0833	
B D,E,G,H,I,J C E I 0 b,d,f,h,i 0 0	0,0283	0,0848	

PRUEBAS BIPARTICIONES			
ONodes			
Partición		Pérdida	Tiempo
A,B,C,D,E,F,G,H,I,J,K,L,M,O N a,b,c,d,e,f,g,h,i,j,k,l,m,n,o ∅		0,0037	3,2139
A B,C,D,E,F,G,H,I,J,K,L,M,N,O ∅ a,b,c,d,e,f,g,h,i,j,k,l,m,n		0,0062	2,3280
A,B,C,D,E,F,G,H,I,J,K,L,M,N,O E b,c,d,e,f,g,h,i,j,k,l,m,n,o ∅		0,0200	1,9944
A,B,C,D,E,F,G,H,I,J,K,L,M,N,O I b,c,d,e,f,g,h,i,j,k,l,m,n ∅		0,0084	1,5500
A,B,C,D,E,F,G,H,I,J,K,L,M,O N a,b,d,e,g,h,i,j,k,m,n ∅		0,0127	1,1393
A,B,C,D,E,F,G,H,I,J,L,M,N,O K a,c,e,g,i,k,m,o ∅		0,0015	0,7682
A,B,C,D,E,F,G,I,J,K,L,M,N,O H b,d,f,h,j,l,n ∅		0,0015	0,6210
A,B,C,D,E,F,G,H,I,J,K,L,M N a,b,c,d,e,f,g,h,i,j,k,l,m,n,o ∅		0,0037	2,9872
A B,C,D,E,F,G,H,I,J,K,L,M,N ∅ a,b,c,d,e,f,g,h,i,j,k,l,m,n		0,0062	2,1554
A,B,C,D,E,F,G,H,I,J,K,L,M,N E b,c,d,e,f,g,h,i,j,k,l,m,n,o ∅		0,0200	1,6228
A,B,C,D,E,F,G,H,I,J,K,L,M,N I b,c,d,e,f,g,h,i,j,k,l,m,n ∅		0,0084	1,3322
A,B,C,D,E,F,G,H,I,J,K,L,M N a,b,d,e,g,h,i,j,k,m,n ∅		0,0127	0,9595
A,B,C,D,E,F,G,H,I,J,L,M,N K a,c,e,g,i,k,m,o ∅		0,0015	0,6043
A,B,C,D,E,F,G,I,J,K,L,M,N H b,d,f,h,j,l,n ∅		0,0015	0,4980
B,C,D,E,F,G,H,I,J,K,L,M,O N a,b,c,d,e,f,g,h,i,j,k,l,m,n,o ∅		0,0037	3,0266
B,C,D,E,F,H,I,J,K,L,M,N,O G a,b,c,d,e,f,g,h,i,j,k,l,m,n ∅		0,0084	2,3892
B,C,D,E,F,G,H,I,J,K,L,M,N,O E b,c,d,e,f,g,h,i,j,k,l,m,n,o ∅		0,0200	1,6997
B,C,D,E,F,G,H,I,J,K,L,M,N,O I b,c,d,e,f,g,h,i,j,k,l,m,n ∅		0,0084	1,5600
B,C,D,E,F,G,H,I,I,K,L,M,O N a,b,d,e,g,h,i,k,m,n ∅		0,0127	0,8820

PRUEBAS BIPARTICIONES			
ONodes			
Partición		Pérdida	Tiempo
A,B,C,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T D a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t ∅		0,0055	262,3030
A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,Q,R,S,T P a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s ∅		0,0170	44,1787
A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,S,T R b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t ∅		0,0018	86,8351
A,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T B b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s ∅		0,0002	29,8275
A,B,C,D,E,F,G,H,I,K,L,M,N,O,P,Q,R,S,T I a,b,d,e,g,h,i,j,k,m,n,p,q,s,t ∅		0,0112	4,7244
A,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T B a,c,e,g,i,k,m,o,q,s ∅		0,0008	1,7545
A,B,C,D,E,F,G,H,I,J,K,L,M,N,P,Q,R,S,T O b,d,f,h,j,l,n,p,r,t ∅		0,0002	1,6116
A,B,C,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S D a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t ∅		0,0055	271,2906
A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,Q,R,S P a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s ∅		0,0170	37,8922
A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,S R b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t ∅		0,0018	79,0286
A,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S B b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s ∅		0,0002	29,0104
A,B,C,D,E,F,G,H,I,K,L,M,N,O,P,Q,R,S I a,b,d,e,g,h,i,j,k,m,n,p,q,s,t ∅		0,0112	3,8910
A,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S B a,c,e,g,i,k,m,o,q,s ∅		0,0008	1,4568
A,B,C,D,E,F,G,H,I,J,K,L,M,N,P,Q,R,S O b,d,f,h,j,l,n,p,r,t ∅		0,0002	1,3360
B,C,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T D a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t ∅		0,0055	199,2890
B,C,D,E,F,G,H,I,J,K,L,M,N,O,Q,R,S,T P a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s ∅		0,0170	35,1517
B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,S,T R b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t ∅		0,0018	76,5596
B C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T ∅ b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s		0,0002	31,2014
B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T I a,b,d,e,g,h,i,k,m,n,p,q,s,t ∅		0,0112	4,0153

11. Uso de Inteligencia Artificial Generativa.

Durante el ciclo de vida del proyecto, se emplearon herramientas de Inteligencia Artificial generativa como asistentes de soporte, específicamente en las fases de entendimiento del problema, correcciones y adiciones al código que vimos pertinentes. El proyecto se hizo con la base planteada por el repositorio compartido de la profesora.

Las herramientas que llegamos a usar fueron Google Gemini, ChatGPT. Estas herramientas las usamos principalmente para entender los documentos del proyecto que se compartieron, porque al tener tanto tecnicismo y palabras complejas y el poco contexto que teníamos sobre las cosas, estas herramientas fueron muy útiles para profundizar el entendimiento del problema y saber el porque iba cada cosa y como funciona la estrategia geométrica que se plantean.

Los prompts fueron hechos de manera muy simple buscando entender partes específicas de las cosas, como por ejemplo conceptos como la marginalización, que eran las TPM' s, que tenían que hacer explicaciones de fórmulas matemáticas que encontrábamos en el problema, saber que se estaba logrando esa estrategia GeoMIP.

Ejecutamos prompts como los que veremos a continuación:

La Matriz de Probabilidad de Transición (TPM) quiero que me expliques mucho mejor, porque siento que hay mucho tecnicismo en este documento, si puedes utilizar ejemplos que me ayuden a comprender mejor, haciendo uso de analogías o cosas simples para la explicación.

En este punto le estoy preguntando más a fondo cómo funcionaban las TPM' s del algoritmo en el documento, ya que no teníamos la claridad de cómo se manejaban en el proyecto, ya que la base de código que nos habían dado las matrices funcionaban muy distintas, y era por el tema de la optimización de la estrategia geométrica que usaban las matrices $2^n * n$ y el documento se hacían $2^n * 2^n$ Pero eso era porque después se le hacía el proceso para cambiarlo a estado-nodo y no a estado a estado, y esto no se manejaba de esa manera porque era imposible almacenar matrices con tantos nodos al tiempo por lo que había que hacer esa conversión.

Continuando con los prompts, queríamos saber a raíz de esas matrices cuál era ese término de la marginalización, y porque se utilizaba tanto del proyecto, se le hacían preguntas como:

Pasando algo que también se especifica mucho en el proyecto, ese término de la marginalización que significa dentro del proyecto, porque lo utilizamos, que estamos logrando con él, realmente es algo que debemos tener en cuenta dentro del proyecto?

Este prompt como vemos fue también realizado para preguntar acerca de ese término, ya que no nos quedaba muy claro qué era lo que hacía dentro de ese proyecto y la función que cumplía, la IA nos daba una respuesta por medio de analogías con unas bombillas, era que como ignorar, por lo que separábamos alguno de los nodos para no tenerlo en cuenta,

entonces estas respuestas nos ayudaban a entender de manera global el contexto, y nos ponemos a mirar después en el documento y descubrimos que tenía sentido lo que nos mencionaba.

Otro Prompt Utilizado fue para entender el tema del hipercubo y eso cómo se manejaba dentro del problema, porque era un tema que nunca habíamos escuchado antes, por lo que se nos dificultó mucho al inicio entender este concepto, porque sentíamos que el documento lo mencionaba de una manera muy superficial y sin entrar tanto a detalles, y nos extrañaba porque en el proyecto se mencionaba muchas palabras como n-cubes, por lo que usábamos prompts para entender más claramente este término

Que significa el término de hipercubo, necesitamos que nos expliques mediante analogías o ejemplos simples, para entender qué significa mas a fondo, y poder comprender porque se encuentra en la base de código que nos ofreció la profesora, es un tema que tocan superficialmente en el texto y se me hace curioso que eso suceda, puedes explicar con más claridad esto?.

Entonces mediante este prompt comprendimos lo que pretendían esos n-cubos en el proyecto y cómo funcionaban. nos mencionaba que era la forma geométrica que toman las probabilidades de tu red. La matriz TPM en crudo, que era solamente una tabla de números, pero el hipercubo le da una estructura espacial. Entonces funcionaba como un mapa que nos permitía visualizar todos los estados posibles, y que permitía hacer los cortes de una forma más eficiente.

Entonces fueron prompts muy utilizados para la comprensión y entendimiento del proyecto, ya que siento que es más importante entender el porqué de las cosas, que simplemente ponernos a codificar sin más, era un proyecto que nos requería entender muy a profundidad un montón de conceptos por lo que necesitábamos si o si, entender el porqué las cosas funcionaban de esa manera y porque el código base hacía lo que hacía.

Ya por la parte de código utilizamos la herramienta de Claude, está más con la finalidad de asegurarnos de que íbamos por buen camino, y que estuviéramos cumpliendo con lo que pedía el proyecto, ya que de pronto al entender la generalidad de pronto se nos pasaban de largo cosas específicas, la usábamos constante con prompts si estamos cumpliendo los estándares también nos aconsejaba con temas sobre la optimización del código. Porque utilizar la paralelización, era útil, porque necesitábamos usar los chunks para la creación de las matrices TPM' s. etc. Estos prompts fueron varios ejemplos que usamos para la corrección constante del proyecto.

Hasta el momento como se lleva el código de la estrategia de GeoMIP basandote en los documentos del proyecto, que elementos nos faltan para cumplir con las exigencias presentadas por la profesora y qué nos recomiendas para continuar realizando el proyecto.

Utilizando Claude se le pidió constantemente de que se asegurara de lo que llevamos estuviera cumpliendo estrictamente lo que se estaba realizando, para evitar no hacer pasos en falso, y evitar después de hacer correcciones muchos mayores, se le consultaba constantemente, si lo que habíamos hecho estaba bien y que cosas faltaban.

Porque sale este error, donde parece que se desborde la memoria aplicando este sistema en ese momento de las pruebas que se estaban realizando al ejecutar el programa, nos puedes guiar para corregir este error y hacer que el proyecto se ejecute de manera correcta.

Claude nos ayudó enormemente, a la corrección de errores con los códigos, muchas veces no entendemos de donde salían determinados errores, cuando se ejecutaba la aplicación así que utilizabamos Claude para que nos orientara para corregir esas partes donde fallará para poder hacer las pruebas de los sistemas.

Qué nos recomiendas para hacer optimizaciones al código, en el sentido de que una computadora con recursos más bajos pueda llegar a ejecutarlo sin tanto problema, porque a pesar de tener una pc buena, muchas veces se nota que nos falta más optimización para lograr los resultados esperados.

Por último se le consultó mucho relacionado a las optimizaciones pertinentes que deberíamos hacer sobre el código que habíamos hecho, nos recomendó mucho sobre el uso de los Chunks para hacer las TPM' s mas grandes, tambien el uso de la paralelización para aprovechar mejor la computadora y no saturar tanto en una evaluación, también se hizo el uso de memorias como si fuera programación dinámica para almacenar resultados de evaluaciones que se repetían constantemente, y recuperar el valor ya obtenido de antes y así obtener los resultados más rápidamente.

Estos fueron los principales prompts que utilizamos con el uso de las 3 IA' s que se mencionaron, fueron aliadas que nos permitieron hacer correcciones y mejorar el proyecto constantemente y lo más importante entender su funcionamiento para saber que era lo que se debía hacer en el proyecto y poder terminarlo.

Por último quiero hacer hincapié en unos últimos detalles, aunque la IA puede parecer algo malo porque tiende a hacer las cosas por sí sola, hay que decir que cuando se utiliza de manera consciente, puede ser un aliado muy importante para nosotros los seres humanos, y más con cosas tan complejas que muchas veces nos cuesta entender, la IA ayuda mucho a ver nuestros fallos y ayudarnos a comprender mejor las cosas.

Aunque la IA ha avanzado mucho con el pasar de los años, hay que saber que es compleja contextualizarla con un proyecto como estos, es algo bastante complejo, porque incluso para obtener explicaciones sobre las cosas se confunde y menciona ejemplos o explicaciones que se salen de lo que realmente necesitábamos saber, por lo que entrenarla para que supiera realizar las peticiones que se le pedían también es algo que se tenía que trabajar, por lo que a pesar de que la IA puede ser una aliada importante, para nosotros, el uso responsable de ella y saberla utilizar es algo que es muy importante para lograr los mejores resultados.